

Exercise: From Linear Regression to Logistic Regression

Dương Đình Thắng

Nguyễn Đặng Duy Tân

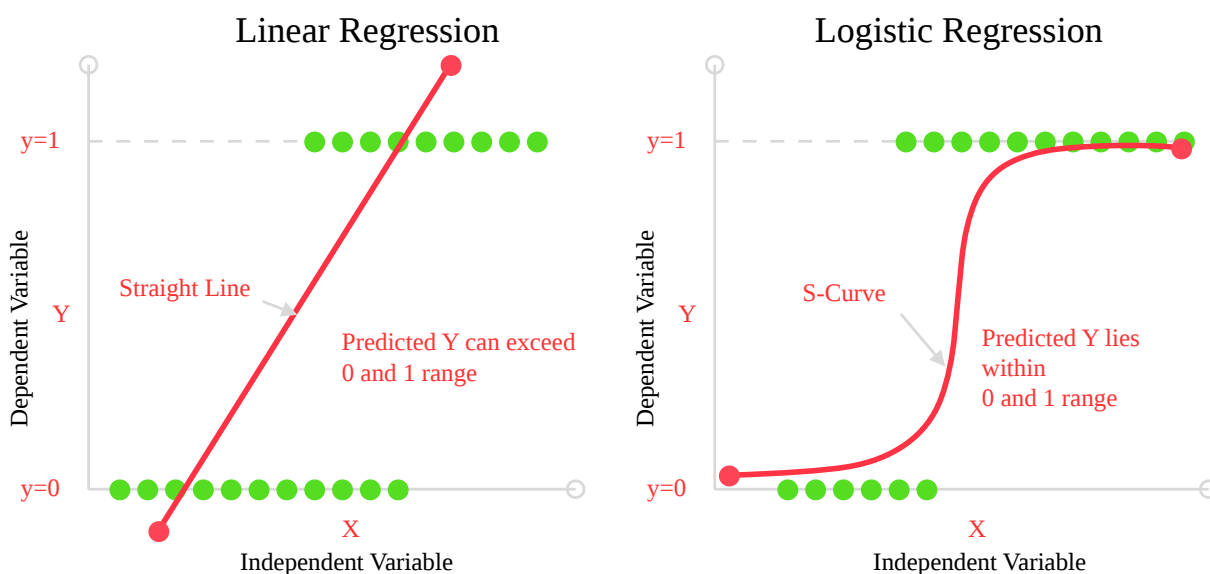
Đình Quang Vinh

1. Giới thiệu

Linear Regression (tạm dịch: hồi quy tuyến tính) là mô hình dự đoán một giá trị liên tục từ các đặc trưng đầu vào, dựa trên giả định quan hệ giữa chúng xấp xỉ tuyến tính. Phương pháp bắt nguồn từ kỹ thuật bình phương tối thiểu, do Legendre công bố năm 1805 [1] và Gauss trình bày năm 1809 [2]. Thuật ngữ regression được Francis Galton đặt năm 1886 [3] khi mô tả hiện tượng hồi quy về trung bình trong nghiên cứu di truyền.

Tuy nhiên, không phải bài toán nào cũng có đầu ra liên tục. Khi nhận chỉ nhận một trong hai giá trị, chẳng hạn thư rác hay không phải thư rác, khối u lành tính hay ác tính, sinh viên đậu hay trượt, ta có bài toán phân loại nhị phân. Đầu ra của hàm tuyến tính là một số thực bất kỳ nên không diễn giải được thành xác suất. Logistic Regression (tạm dịch: hồi quy logistic) khắc phục hạn chế này bằng cách đưa đầu ra tuyến tính qua hàm Sigmoid.

Trong bài viết này, chúng ta sẽ đi từ Linear Regression đến Logistic Regression, xây dựng từng bước quy trình huấn luyện của hai mô hình.



Hình 1: Hình ảnh minh họa về Linear Regression và Logistic Regression.

Mục lục

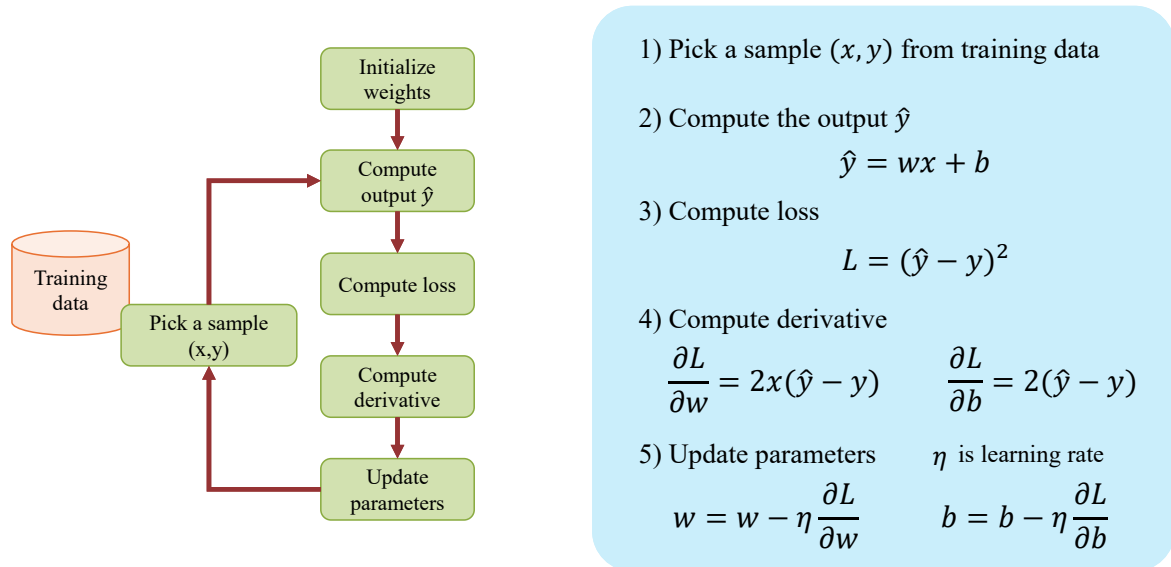
1.	Giới thiệu	1
2.	Các bước trong Linear Regression cho bài toán hồi quy	4
3.	Chuẩn bị dữ liệu	5
4.	Huấn luyện mô hình	6
5.	Chuẩn hoá đặc trưng	9
5.1.	Min-Max Scaling	9
5.2.	Inverse Min-Max Scaling	9
5.3.	Z-score Scaling (Standardization)	10
5.4.	Inverse Z-score Scaling	10
6.	Vector hoá mô hình Linear Regression	12
6.1.	Tổng quan	12
6.2.	Linear Regression áp dụng từng mẫu dữ liệu	13
6.3.	Linear Regression cho batch (mini-batch hoặc full batch)	14
7.	Xây dựng mô hình Linear Regression cho bài toán advertising	15
7.1.	Stochastic Gradient Descent	16
7.2.	Mini-batch Gradient Descent	18
7.3.	Batch Gradient Descent	20
8.	Xây dựng mô hình Linear Regression cho bài toán dự đoán Bitcoin	22
8.1.	Bước 1: Tải và đọc dữ liệu	22
8.2.	Bước 2: Khám phá dữ liệu	23
8.3.	Bước 3: Tạo biểu đồ nền giá giao dịch từ năm 2019 - 2022	24
8.4.	Bước 4: Tạo mô hình Linear Regression	25
8.5.	Bước 5: Chia tập train và test	26
8.6.	Bước 6: Huấn luyện và đánh giá mô hình tự xây dựng	26
8.7.	Bước 7: Huấn luyện và đánh giá mô hình từ thư viện scikit-learn	29
9.	Từ Linear Regression đến Logistic Regression	32
9.1.	Bài toán phân loại nhị phân	32
9.2.	Thuật ngữ và ký hiệu	33
9.3.	Ôn tập nhanh Linear Regression	35

9.4.	Hàm Sigmoid	36
9.5.	Hàm loss mới: Binary Cross-Entropy	39
9.6.	Gradient Descent cho Logistic Regression	42
9.7.	Quy trình huấn luyện	45
10.	Thực hành Logistic Regression	47
10.1.	Phát biểu bài toán	47
10.2.	Chi tiết tính toán	47
10.3.	Phân tích kết quả epoch 1	49
10.4.	Phân tích kết quả huấn luyện qua các epoch	50
11.	Xây dựng mô hình Logistic Regression cho bài toán Titanic	52
11.1.	Bước 1: Tải và đọc dữ liệu	52
11.2.	Bước 2: Khám phá dữ liệu	53
11.3.	Bước 3: Tách X, y và thêm cột bias	53
11.4.	Bước 4: Chia tập train, validation và test	53
11.5.	Bước 5: Chuẩn hoá đặc trưng	54
11.6.	Bước 6: Huấn luyện mô hình	54
11.7.	Bước 7: Đánh giá mô hình	57
12.	Tài liệu tham khảo	59
13.	Phụ lục	60

2. Các bước trong Linear Regression cho bài toán hồi quy

Xuyên suốt phần Linear Regression, chúng ta dùng chung bộ dữ liệu advertising.csv, trong đó ba đặc trưng đầu vào là chi phí quảng cáo trên TV, Radio, Newspaper và nhãn cần dự đoán là doanh số Sales.

Các bước huấn luyện dưới đây minh hoạ cho trường hợp ba đặc trưng này, với tham số được cập nhật theo từng mẫu (Stochastic Gradient Descent).



Hình 2: Quá trình huấn luyện mô hình hồi quy tuyến tính.

Step 0: Khởi tạo trọng số

Để huấn luyện mô hình chúng ta khởi tạo các trọng số gồm w và b .

Step 1: Lấy một sample

Chọn một mẫu (x_1, x_2, x_3, y) từ tập huấn luyện.

Step 2: Tính output dự đoán

$$\hat{y} = w_1x_1 + w_2x_2 + w_3x_3 + b.$$

Step 3: Tính hàm loss

$$L = (\hat{y} - y)^2.$$

Step 4: Tính đạo hàm (gradient)

$$\frac{\partial L}{\partial w_1} = 2x_1(\hat{y} - y), \quad \frac{\partial L}{\partial w_2} = 2x_2(\hat{y} - y), \quad \frac{\partial L}{\partial w_3} = 2x_3(\hat{y} - y), \quad \frac{\partial L}{\partial b} = 2(\hat{y} - y).$$

Step 5: Cập nhật tham số

$$w_1 \leftarrow w_1 - \eta \frac{\partial L}{\partial w_1}, \quad w_2 \leftarrow w_2 - \eta \frac{\partial L}{\partial w_2}, \quad w_3 \leftarrow w_3 - \eta \frac{\partial L}{\partial w_3}, \quad b \leftarrow b - \eta \frac{\partial L}{\partial b}.$$

Sau khi cập nhật, quay lại **Step 1** và tiếp tục lặp qua các mẫu cho đến khi hội tụ hoặc đạt số vòng lặp tối đa.

3. Chuẩn bị dữ liệu

Sau khi đã nắm các bước ở dạng công thức, ta bắt đầu cài đặt bằng Python, khởi đầu là khâu đọc dữ liệu.

Đoạn code bên dưới đã tải sẵn file advertising.csv.

Yêu cầu 1: Hoàn thành hàm `get_column(data, index)` để lấy ra cột thứ `index` của dữ liệu. Hàm `prepare_data(file_name_dataset)` cho sẵn bên dưới sẽ dùng nó để trả về X (đầu vào) cùng y (đầu ra).

```

1 !gdown 1GmYeNtWTvah2VHL1EPSTmdfbMqfjFITc
2 !unzip advertising.zip
3
4 import numpy as np
5 import matplotlib.pyplot as plt
6 import random
7
8 def get_column(data, index):
9
10     ***your code here***
11
12     return result
13
14 def prepare_data(file_name_dataset):
15     data = np.genfromtxt(file_name_dataset, delimiter=',', skip_header=1).tolist()
16     N = len(data)
17
18     # get tv (index=0)
19     tv_data = get_column(data, 0)
20
21     # get radio (index=1)
22     radio_data = get_column(data, 1)
23
24     # get newspaper (index=2)
25     newspaper_data = get_column(data, 2)
26
27     # get sales (index=3)
28     sales_data = get_column(data, 3)
29
30     # building X input and y output for training
31     X = [tv_data, radio_data, newspaper_data]
32     y = sales_data
33     return X,y

```

4. Huấn luyện mô hình

Ta dùng X và y thu được ở phần trên để huấn luyện mô hình dự đoán **sales**, với tham số được cập nhật theo từng mẫu (one-sample training).

Loss được tính bằng công thức Mean Squared Error:

$$L = (\hat{y} - y)^2$$

Trong thực tế, các tham số nên được khởi tạo ngẫu nhiên. Tuy nhiên, để thống nhất kết quả đánh giá cho toàn bộ bài tập trong module này, các bạn khởi tạo cố định w_1, w_2, w_3 và b như sau:

```
1 def initialize_params():
2     # w1 = random.gauss(mu=0.0, sigma=0.01)
3     # w2 = random.gauss(mu=0.0, sigma=0.01)
4     # w3 = random.gauss(mu=0.0, sigma=0.01)
5     # b = 0
6
7     w1, w2, w3, b = (0.016992259082509283, 0.0070783670518262355, -0.002307860847821344, 0)
8     return w1, w2, w3, b
```

≡ **Yêu cầu 2:** Hoàn thành hàm **predict(x1, x2, x3, w1, w2, w3, b)** để trả về giá trị dự đoán $\hat{y}$ tương ứng.

```
1 def predict(x1, x2, x3, w1, w2, w3, b):
2
3     ***your code here***
4
5     return result
```

≡ **Yêu cầu 3:** Hoàn thành hàm **compute_loss(y_hat, y)** để tính loss giữa giá trị dự đoán y_{hat} và giá trị thực y , dùng Mean Squared Error.

```
1 def compute_loss(y_hat, y):
2
3     ***your code here***
4
5     return result
```

≡ **Yêu cầu 4:** Hoàn thành hàm **compute_gradient_wi(xi, y, y_hat)** để tính đạo hàm của hàm loss

$$L = (\hat{y} - y)^2$$

theo w_i và function **compute_gradient_b(y, y_hat)** để tính đạo hàm của hàm loss L theo b .

```

1 def compute_gradient_wi(xi, y, y_hat):
2
3     ***your code here***
4
5     return dl_dwi
6
7 def compute_gradient_b(y, y_hat):
8
9     ***your code here***
10
11    return dl_db

```

☰ **Yêu cầu 5:** Hoàn thành hàm `update_weight_wi(wi, dl_dwi, lr)` để cập nhật w_i sau khi tính đạo hàm loss L theo w_i , và function `update_weight_b(b, dl_db, lr)` để update bias (b) sau khi tính đạo hàm loss L theo b .

```

1 def update_weight_wi(wi, dl_dwi, lr):
2
3     ***your code here***
4
5     return wi
6
7 def update_weight_b(b, dl_db, lr):
8
9     ***your code here***
10
11    return b

```

Sau khi hoàn thành các function trên ta sẽ tạo được function `implement_linear_regression(X_data, y_data, epoch_max, lr)` và trả về 4 tham số w_1, w_2, w_3, b cùng lịch sử tính loss như bên dưới:

```

1 def implement_linear_regression(X_data, y_data, epoch_max = 50, lr = 1e-5):
2     losses = []
3
4     w1, w2, w3, b = initialize_params()
5
6     N = len(y_data)
7     for epoch in range(epoch_max):
8         for i in range(N):
9             # get a sample
10            x1 = X_data[0][i]
11            x2 = X_data[1][i]
12            x3 = X_data[2][i]
13
14            y = y_data[i]
15
16            # print(y)
17            # compute output
18            y_hat = predict(x1, x2, x3, w1, w2, w3, b)

```

```

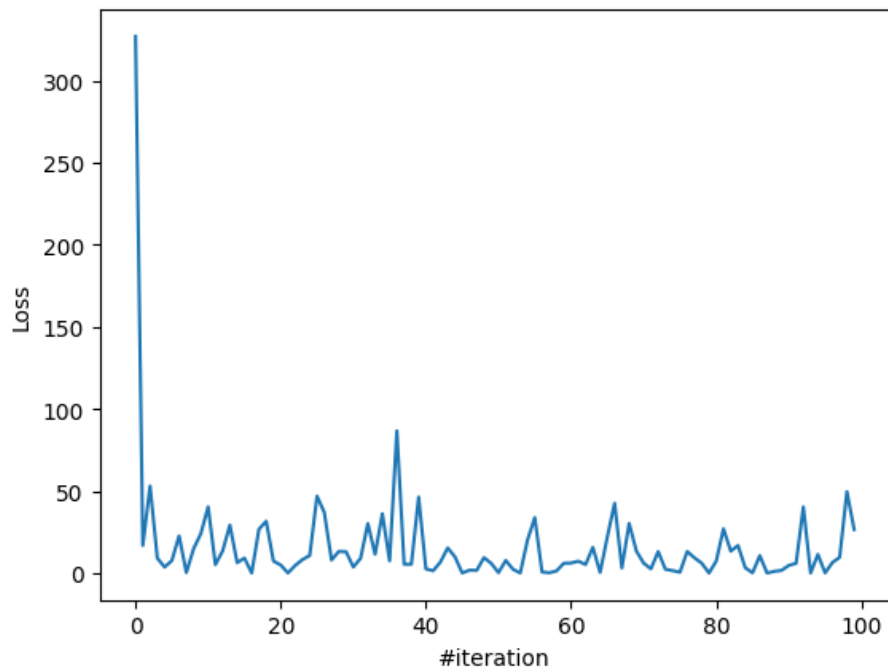
19
20     # compute loss
21     loss = compute_loss(y_hat, y)
22
23     # compute gradient w1, w2, w3, b
24     dl_dw1 = compute_gradient_wi(x1, y, y_hat)
25     dl_dw2 = compute_gradient_wi(x2, y, y_hat)
26     dl_dw3 = compute_gradient_wi(x3, y, y_hat)
27     dl_db = compute_gradient_b(y, y_hat)
28
29     # update parameters
30     w1 = update_weight_wi(w1, dl_dw1, lr)
31     w2 = update_weight_wi(w2, dl_dw2, lr)
32     w3 = update_weight_wi(w3, dl_dw3, lr)
33     b = update_weight_b(b, dl_db, lr)
34
35     # logging
36     losses.append(loss)
37     return (w1,w2,w3,b, losses)

```

```

1 X,y = prepare_data('../content/advertising.csv')
2 (w1,w2,w3,b, losses) = implement_linear_regression(X,y)
3 print(losses[0:100])
4 plt.plot(losses[0:100])
5 plt.xlabel("#iteration")
6 plt.ylabel("Loss")
7 plt.show()

```



Hình 3: Kết quả loss sau 100 iteration cập nhật.

5. Chuẩn hoá đặc trưng

Ba đặc trưng TV, Radio và Newspaper có khoảng giá trị chênh lệch nhau khá nhiều, khiến gradient theo từng tham số có độ lớn rất khác nhau và mô hình khó hội tụ. Chuẩn hoá khắc phục điều này bằng cách đưa các đặc trưng về cùng một thang đo.

Trong phần này, chúng ta sẽ được tìm hiểu thêm về hai phương pháp chuẩn hoá dữ liệu phổ biến: **Min-Max Scaling** và **Z-score Scaling (Standardization)**. Ngoài ra, chúng ta cũng sẽ giới thiệu cách thực hiện **inverse transform** để đưa dữ liệu đã chuẩn hoá trở về giá trị ban đầu. Các ví dụ minh hoạ sẽ giúp làm rõ cơ chế hoạt động của từng phương pháp.

5.1. Min-Max Scaling

Phương pháp này đưa dữ liệu về khoảng $[0, 1]$ theo công thức:

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

trong đó $x_{\min}$ và $x_{\max}$ lần lượt là giá trị nhỏ nhất và lớn nhất của thuộc tính.

5.2. Inverse Min-Max Scaling

Để khôi phục dữ liệu gốc từ dữ liệu đã chuẩn hoá, ta sử dụng:

$$x = x' \times (x_{\max} - x_{\min}) + x_{\min}$$

Min-Max Scaling & Inverse Min-Max Scaling

```
1 # -----
2 # 1. Min-Max Scaling
3 # -----
4 def min_max_scaling(X):
5     mins = []
6     maxs = []
7     X_scaled = []
8
9     # Tìm min và max cho từng feature
10    for feature in X:
11        min_val = min(feature)
12        max_val = max(feature)
13        mins.append(min_val)
14        maxs.append(max_val)
15
16    # Scale từng phần tử
17    scaled_feature = []
18    for x in feature:
19        scaled_x = (x - min_val) / (max_val - min_val)
20        scaled_feature.append(scaled_x)
21
22    X_scaled.append(scaled_feature)
23
24    return X_scaled, mins, maxs
25
26
```

```

27 # -----
28 # 2. Inverse Min-Max
29 # -----
30 def inverse_min_max_scaling(X_scaled, mins, maxs):
31     X_recovered = []
32
33     for i in range(len(X_scaled)):
34         feature = X_scaled[i]
35         min_val = mins[i]
36         max_val = maxs[i]
37
38         recovered_feature = []
39         for x in feature:
40             original_x = x * (max_val - min_val) + min_val
41             recovered_feature.append(original_x)
42
43         X_recovered.append(recovered_feature)
44
45     return X_recovered

```

5.3. Z-score Scaling (Standardization)

Phương pháp này chuẩn hoá dữ liệu về phân phối có trung bình bằng 0 và độ lệch chuẩn bằng 1:

$$x' = \frac{x - \mu}{\sigma}$$

trong đó μ là giá trị trung bình và σ là độ lệch chuẩn.

5.4. Inverse Z-score Scaling

Để đưa dữ liệu đã chuẩn hoá về ban đầu, ta áp dụng:

$$x = x' \times \sigma + \mu$$

Z-score Scaling (Standardization) & Inverse Z-score Scaling

```

1 # -----
2 # 3. Z-Score Scaling
3 # -----
4 def z_score_scaling(X):
5     means = []
6     stds = []
7     X_scaled = []
8
9     for feature in X:
10         mean_val = sum(feature) / len(feature)
11         means.append(mean_val)
12
13         std_val = (sum((x - mean_val) ** 2 for x in feature) / (len(feature) - 1)) ** 0
14                     .5
15
16         stds.append(std_val)
17
18         scaled_feature = []

```

```
17         for x in feature:
18             scaled_x = (x - mean_val) / std_val
19             scaled_feature.append(scaled_x)
20
21         X_scaled.append(scaled_feature)
22
23     return X_scaled, means, stds
24
25
26     # -----
27     # 4. Inverse Z-Score
28     # -----
29 def inverse_z_score_scaling(X_scaled, means, stds):
30     X_recovered = []
31
32     for i in range(len(X_scaled)):
33         feature = X_scaled[i]
34         mean_val = means[i]
35         std_val = stds[i]
36
37         recovered_feature = []
38         for x in feature:
39             original_x = x * std_val + mean_val
40             recovered_feature.append(original_x)
41
42         X_recovered.append(recovered_feature)
43
44     return X_recovered
```

Ví dụ áp dụng

```
1 data1 = [1, 2, 3]
2 data2 = [4, 5, 6]
3 data3 = [7, 8, 9]
4 X = [data1, data2, data3]
5
6 # Min-Max
7 X_minmax, mins, maxs = min_max_scaling(X)
8 print("Min-Max Scaled:", X_minmax)
9 print("Recovered Min-Max:", inverse_min_max_scaling(X_minmax, mins, maxs))
10
11 # Z-Score
12 X_zscore, means, stds = z_score_scaling(X)
13 print("\nZ-Score Scaled:", X_zscore)
14 print("Recovered Z-Score:", inverse_z_score_scaling(X_zscore, means, stds))
```

6. Vector hoá mô hình Linear Regression

Các phần trên thao tác trên từng biến rời rạc. Từ đây, ta biểu diễn dữ liệu dưới dạng vector và ma trận để tận dụng các phép toán vector hoá của NumPy, nhờ đó rút gọn công thức và giảm số vòng lặp trong cài đặt. Bảng dưới đây quy định các ký hiệu toán học dùng cho phần còn lại của tài liệu.

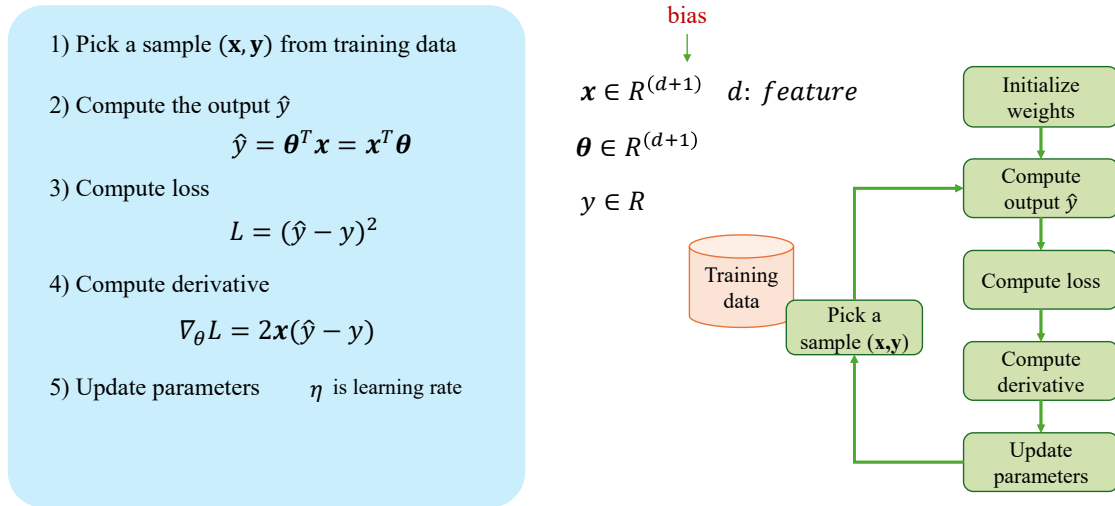
Bảng Ký hiệu Toán học

Ký hiệu	Ý nghĩa
x_i	Đặc trưng (đầu vào) của mẫu thứ i (trường hợp 1 biến).
y_i	Nhãn (giá trị mục tiêu) của mẫu i .
$\hat{y}_i$	Giá trị dự đoán từ x_i .
w, b	Trọng số và bias của mô hình.
ε_i	Sai số ngẫu nhiên (nhiều) giữa y_i và $\hat{y}_i$.
d	Số đặc trưng.
m	Số mẫu trong một batch. Khi dùng full-batch, m bằng kích thước toàn tập dữ liệu.
$\mathbb{R}^{a \times b}$	Không gian các ma trận số thực kích thước $a \times b$ (a hàng, b cột).
$\mathbf{x} \in \mathbb{R}^{(d+1)}$	Vector đặc trưng đã gộp bias cho một mẫu.
$\boldsymbol{\theta} \in \mathbb{R}^{(d+1)}$	Vector tham số (gồm các w_j và b).
$\mathbf{X} \in \mathbb{R}^{m \times (d+1)}$	Ma trận đặc trưng của một batch m mẫu (đã gộp bias).
$\mathbf{y} \in \mathbb{R}^m$	Vector mục tiêu của batch.
L	Hàm loss, đo độ sai lệch giữa dự đoán và thực tế.
$\frac{\partial L_i}{\partial w}, \frac{\partial L_i}{\partial b}$	Thành phần gradient theo từng tham số tại mẫu i .
$\nabla_{\boldsymbol{\theta}} L$	Gradient, xác định hướng và tốc độ dốc nhất để giảm lỗi.
η	Learning rate, quyết định độ lớn của mỗi bước cập nhật tham số.

6.1. Tổng quan

Mục tiêu của Linear Regression là mô hình hoá quan hệ gần tuyến tính giữa đầu vào và đầu ra để dự đoán hoặc giải thích ảnh hưởng của từng biến đầu vào lên kết quả.

Trong bài toán có d đặc trưng x và nhãn y , mô hình dự đoán cho từng mẫu dữ liệu theo công thức $\hat{y} = \mathbf{x}^T \boldsymbol{\theta}$, trong đó $\mathbf{x}$ là vector đặc trưng đã gộp bias và $\boldsymbol{\theta}$ là vector tham số. Hình dưới đây tóm tắt các bước huấn luyện trên một mẫu.



Hình 4: Hình ảnh minh họa về quá trình huấn luyện Linear Regression với 1 sample.

6.2. Linear Regression áp dụng từng mẫu dữ liệu

Step 1: Khởi tạo

Khởi tạo tham số đã gộp bias:

$$\boldsymbol{\theta} = \begin{bmatrix} w_1 \\ \vdots \\ w_d \\ b \end{bmatrix} \in \mathbb{R}^{(d+1)}, \quad \boldsymbol{\theta} \sim \mathcal{N}(0, 0.01^2 I_{d+1}).$$

Chú ý: Chúng ta hoàn toàn có thể đặt b đứng trước w_1 trong vector $\boldsymbol{\theta}$, thay vì đứng sau w_d như trên. Lúc này, vector $\mathbf{x}$ ở bước sau cũng cần thay đổi vị trí số 1 tương ứng.

Step 2: Lấy một mẫu

Chọn một cặp (x, y) với

$$\mathbf{x} = \begin{bmatrix} x_1 \\ \vdots \\ x_d \\ 1 \end{bmatrix} \in \mathbb{R}^{(d+1)}, \quad y \in \mathbb{R}.$$

Step 3: Dự đoán (vector hoá)

$$\hat{y} = \mathbf{x}^T \boldsymbol{\theta} = \boldsymbol{\theta}^T \mathbf{x} \in \mathbb{R}.$$

Step 4: Loss từng mẫu (MSE)

$$L = (\hat{y} - y)^2.$$

Step 5: Gradient theo vector tham số

$$\nabla_{\boldsymbol{\theta}} L = 2\mathbf{x}(\hat{y} - y) \in \mathbb{R}^{(d+1)}.$$

Step 6: Cập nhật SGD (dùng 1 mẫu mỗi bước, learning rate η)

$$\theta \leftarrow \theta - \eta \nabla_{\theta} L$$

Lặp lại từ **Step 2** qua các mẫu cho đến khi hội tụ hoặc đạt số vòng lặp tối đa.

6.3. Linear Regression cho batch (mini-batch hoặc full batch)

N = total number of training samples
 m = batch size (number of selected samples), $1 < m \leq N$

1) Pick m samples (X, y) from training data

2) Compute the output $\hat{y}$

$$\hat{y} = X\theta$$

3) Compute loss

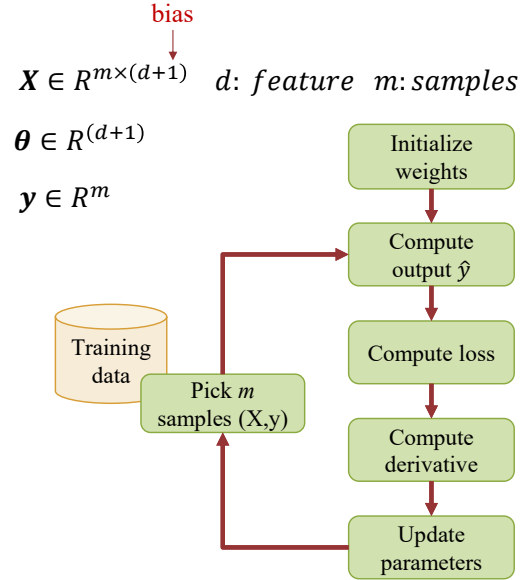
$$L = \frac{1}{m} (\hat{y} - y)^T (\hat{y} - y)$$

4) Compute derivative

$$\nabla_{\theta} L = \frac{2}{m} X^T (\hat{y} - y)$$

5) Update parameters η is learning rate

$$\theta = \theta - \eta \nabla_{\theta} L$$



Hình 5: Hình ảnh minh họa về quá trình huấn luyện Linear Regression với m mẫu.

Step 1: Khởi tạo

Khởi tạo tham số đã gộp bias:

$$\theta = \begin{bmatrix} w_1 \\ \vdots \\ w_d \\ b \end{bmatrix} \in \mathbb{R}^{(d+1)}, \quad \theta \sim \mathcal{N}(0, 0.01^2 I_{d+1}).$$

Step 2: Lấy một batch

Chọn batch m mẫu $(X^{(i)}, y^{(i)})$ với $i = 1, \dots, m$ và ghép thành:

$$X = \begin{bmatrix} X^{(1)} \\ \vdots \\ X^{(m)} \end{bmatrix} \in \mathbb{R}^{m \times (d+1)}, \quad y = \begin{bmatrix} y^{(1)} \\ \vdots \\ y^{(m)} \end{bmatrix} \in \mathbb{R}^m,$$

trong đó mỗi $X^{(i)} \in \mathbb{R}^{1 \times (d+1)}$ là một vector hàng.

Step 3: Dự đoán cho batch

$$\hat{y} = X\theta \in \mathbb{R}^m.$$

Step 4: Loss cho batch

$$L = \frac{1}{m} \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)})^2 = \frac{1}{m} \|\hat{\mathbf{y}} - \mathbf{y}\|_2^2.$$

Step 5: Gradient cho batch

$$\nabla_{\theta} L = \frac{2}{m} \mathbf{X}^{\top} (\hat{\mathbf{y}} - \mathbf{y}) \in \mathbb{R}^{(d+1)}.$$

Step 6: Cập nhật tham số

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \nabla_{\theta} L.$$

Lặp lại từ **Step 2** đến khi hội tụ hoặc đạt số epoch tối đa.

7. Xây dựng mô hình Linear Regression cho bài toán advertising

Trong phần này, chúng ta sẽ cài đặt Linear Regression cho bộ dữ liệu advertising theo hướng vector hoá, với hai cấu hình batch gồm một mẫu cho mỗi bước cập nhật (Stochastic Gradient Descent) và m mẫu cho mỗi bước cập nhật (Mini-batch Gradient Descent, hoặc Batch Gradient Descent khi m bằng kích thước toàn tập dữ liệu).

Thực hiện huấn luyện mô hình Linear Regression trên tập data advertising.csv theo các yêu cầu sau. Các bạn sẽ dựa trên 3 thông tin đầu vào là TV, Radio, Newspaper để dự đoán Sales.

Giới thiệu về tập data: Data có 200 samples (rows), gồm 4 cột thông tin TV, Radio, Newspaper, và Sales. Đề bài yêu cầu dùng thông tin ở 3 cột đầu tiên (TV, Radio, Newspaper) để dự đoán được cột cuối cùng (Sales) dùng linear regression model.

index	TV	Radio	Newspaper	Sales
0	230.1	37.8	69.2	22.1
1	44.5	39.3	45.1	10.4
2	17.2	45.9	69.3	12.0
3	151.5	41.3	58.5	16.5
4	180.8	10.8	58.4	17.9
5	8.7	48.9	75.0	7.2
6	57.5	32.8	23.5	11.8
7	120.2	19.6	11.6	13.2
8	8.6	2.1	1.0	4.8

Hình 6: Một vài sample data từ dữ liệu quảng cáo advertising.csv.

Code tải dataset được cung cấp ở bên dưới như sau:

```
1 !gdown 1GmYeNtWTvah2VHL1EPSTMdfbMqfjFITc
2 !unzip advertising.zip
```

Để chuẩn hoá data đầu vào, bạn được cung cấp trước function đọc dữ liệu và chuẩn hoá **mean__normalization(X)** như bên dưới:

mean_normalization(X)

```

1 import numpy as np
2 from numpy import genfromtxt
3 import matplotlib.pyplot as plt
4
5 # dataset
6 data = genfromtxt("../content/advertising.csv", delimiter=",", skip_header=1)
7 N = data.shape[0]
8 X = data[:, :3]
9 y = data[:, 3:]
10
11 # Normalize input data by using mean normalizaton
12 def mean_normalization(X):
13     N = len(X)
14     maxi = np.max(X)
15     mini = np.min(X)
16     avg = np.mean(X)
17     X = (X-avg) / (maxi-mini)
18     X_b = np.c_[np.ones((N, 1)), X]
19     return X_b, maxi, mini, avg
20
21 X_b, maxi, mini, avg = mean_normalization(X)

```

Yêu cầu của bài tập này là các bạn lần lượt cài đặt lại giải thuật Linear Regression để dự đoán Sales dựa vào các yêu cầu sau:

7.1. Stochastic Gradient Descent

☞ **Yêu cầu 6:** Hoàn thành hàm `stochastic_gradient_descent()` để huấn luyện data sử dụng Stochastic Gradient Descent. Lưu ý các bạn cần tận dụng tối đa vectorization để hoàn thiện bài tập này.

- **input:** (4 inputs) `X_b`, `y`, `n_epochs`, `learning_rate`
- **output:** `thetas_path`, `losses`

stochastic_gradient_descent()

```

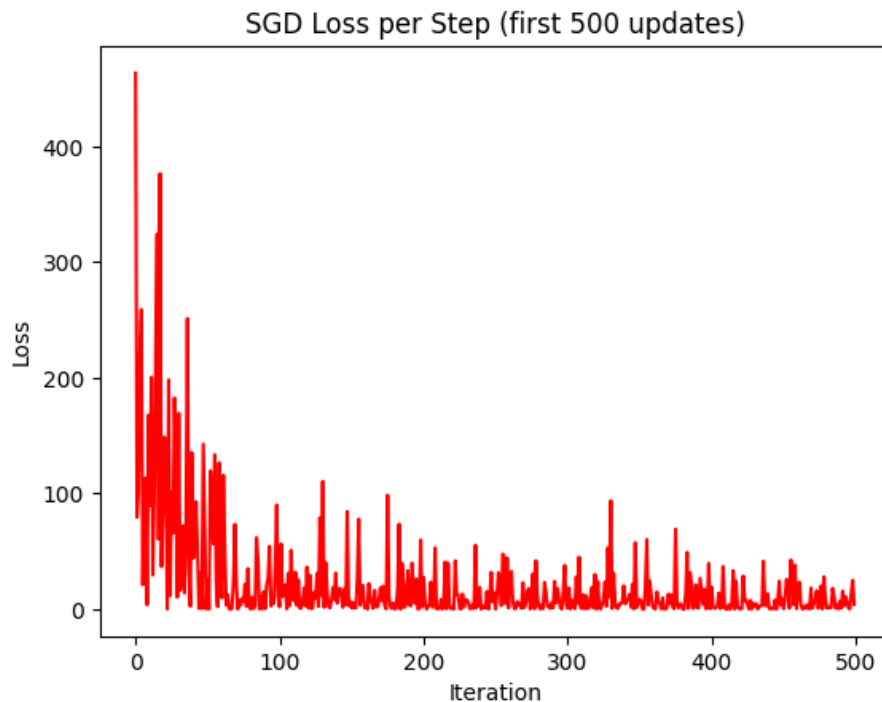
1 def stochastic_gradient_descent(X_b, y, n_epochs=50, learning_rate=0.01):
2     N, d_plus1 = X_b.shape
3
4     # Step 1: Init parameters
5     thetas = np.asarray([[1.16270837], [-0.81960489],
6                          [1.39501033], [0.29763545]])
7     thetas_path = [thetas.copy()]
8     losses = []
9     # Step 2-6: SGD loop
10    for epoch in range(n_epochs):
11        for i in range(N):
12            random_index = i
13            xi = X_b[random_index:random_index+1]
14            yi = y[random_index:random_index+1]
15

```

```
16         # Step 3: prediction
17         ***Your code here***
18
19         # Step 4: MSE loss for 1 sample
20         ***Your code here***
21
22         # Step 5: gradient
23         ***Your code here***
24
25         # Step 6: update
26         ***Your code here***
27
28         # log
29         thetas_path.append(thetas.copy())
30         losses.append(li[0][0])
31     return thetas_path, losses
```

Hình 7 là kết quả sau khi thực thi đoạn code sau:

```
1 sgd_theta, losses = stochastic_gradient_descent(X_b, y, n_epochs=50, learning_rate=0.01
2 )
3 # In loss cho 500 bước đầu
4 x_axis = list(range(500))
5 plt.plot(x_axis, losses[:500], color="r")
6 plt.xlabel("Iteration")
7 plt.ylabel("Loss")
8 plt.title("SGD Loss per Step (first 500 updates)")
9 plt.show()
```



Hình 7: Kết quả loss values sử dụng Stochastic Gradient Descent.

7.2. Mini-batch Gradient Descent

☒ **Yêu cầu 7:** Hoàn thành hàm `mini_batch_gradient_descent()` để huấn luyện data sử dụng Mini-batch Gradient Descent. Lưu ý các bạn cần tận dụng tối đa vectorization để hoàn thiện bài tập này.

- **input:** (5 inputs) `X_b`, `y`, `n_epochs`, `minibatch_size`, `learning_rate`
- **output:** `thetas_path`, `losses`

`mini_batch_gradient_descent()`

```

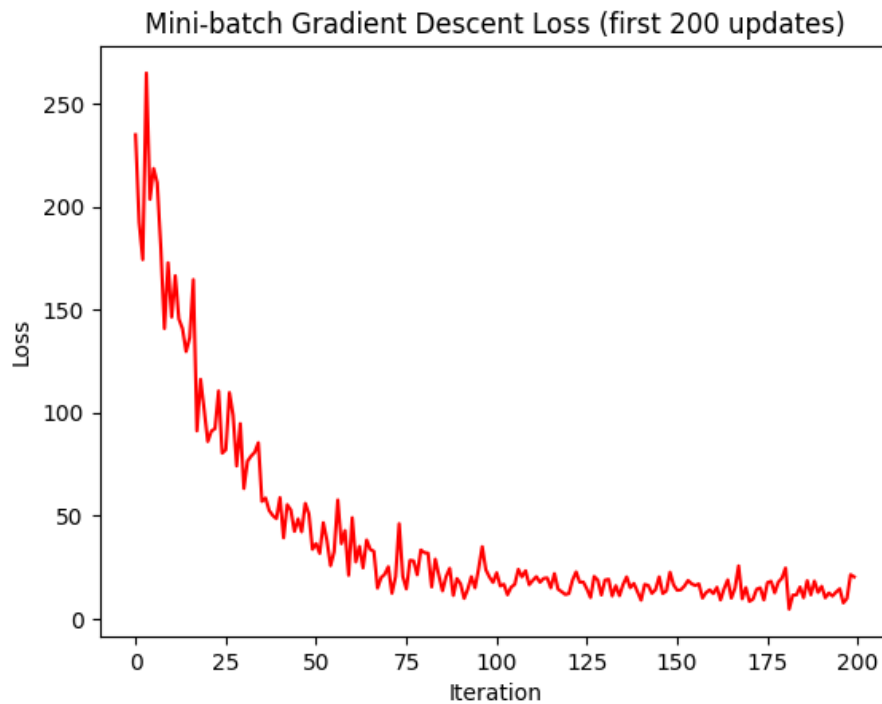
1 def mini_batch_gradient_descent(X_b, y, n_epochs=50, minibatch_size=20, learning_rate=0
    .01):
2     N, d_plus1 = X_b.shape
3     # Step 1: Init parameters
4     thetas = np.asarray([[1.16270837], [-0.81960489],
5                          [1.39501033], [0.29763545]])
6     thetas_path = [thetas.copy()]
7     losses = []
8     for epoch in range(n_epochs):
9         # shuffle data
10        shuffled_indices = np.random.permutation(N)
11        X_b_shuffled = X_b[shuffled_indices]
12        y_shuffled = y[shuffled_indices]
13
14        for i in range(0, N, minibatch_size):
15            xi = X_b_shuffled[i:i+minibatch_size] # (m, d+1)
16            yi = y_shuffled[i:i+minibatch_size]   # (m, 1)
17

```

```
18         # Step 3: prediction
19         ***Your code here***
20
21         # Step 4: MSE loss for minibatch
22         ***Your code here***
23
24         # Step 5: gradient for minibatch
25         ***Your code here***
26
27         # Step 6: update
28         ***Your code here***
29
30         # log
31         thetas_path.append(thetas.copy())
32         losses.append(loss)
33     return thetas_path, losses
```

Hình 8 là kết quả sau khi thực thi đoạn code sau:

```
1 mbgd_thetas, losses = mini_batch_gradient_descent(X_b, y, n_epochs=50, minibatch_size=
2                                     20, learning_rate=0.01)
3 # visualize
4 x_axis = list(range(200))
5 plt.plot(x_axis, losses[:200], color="r")
6 plt.xlabel("Iteration")
7 plt.ylabel("Loss")
8 plt.title("Mini-batch Gradient Descent Loss (first 200 updates)")
9 plt.show()
```



Hình 8: Kết quả loss values sử dụng Mini batch Gradient Descent.

7.3. Batch Gradient Descent

☒ **Yêu cầu 8:** Hoàn thành hàm `batch_gradient_descent()` để huấn luyện data sử dụng batch Gradient Descent. Lưu ý các bạn cần tận dụng tối đa vectorization để hoàn thiện bài tập này.

- **input:** (4 inputs) `X_b`, `y`, `n_epochs`, `learning_rate`
- **output:** `thetas_path`, `losses`

`batch_gradient_descent()`

```

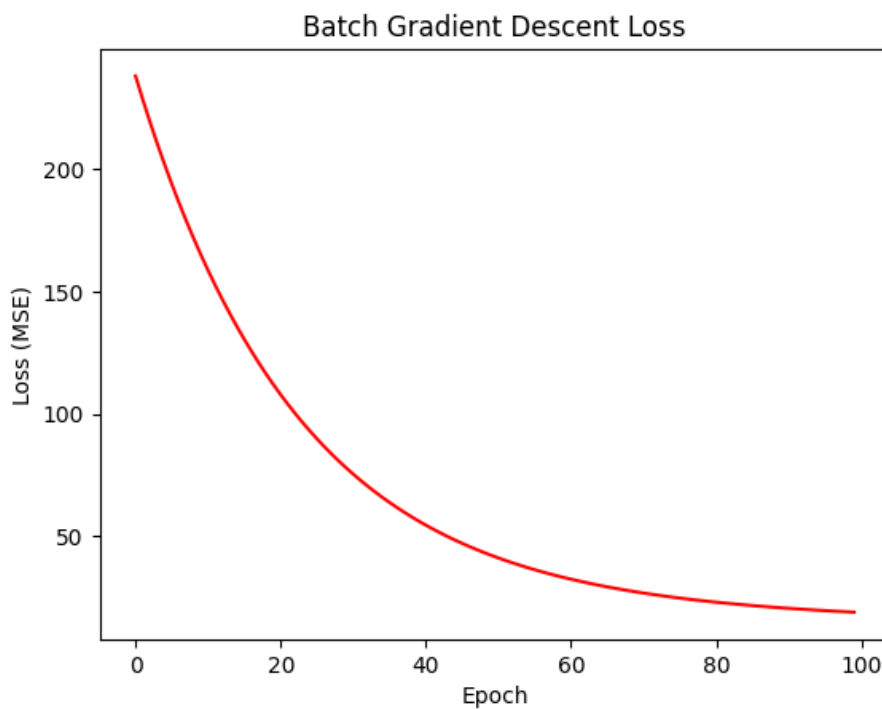
1 def batch_gradient_descent(X_b, y, n_epochs=100, learning_rate=0.01):
2     N, d_plus1 = X_b.shape
3     # Step 1: Init parameters
4     thetas = np.asarray([[1.16270837], [-0.81960489], [1.39501033], [0.29763545]])
5     thetas_path = [thetas.copy()]
6     losses = []
7
8     for epoch in range(n_epochs):
9         # Step 3: prediction
10        ***Your code here***
11
12        # Step 4: MSE loss
13        ***Your code here***
14
15        # Step 5: gradient
16        ***Your code here***
17
18        # Step 6: update parameters

```

```
19     ***Your code here***
20
21     # log
22     thetas_path.append(thetas.copy())
23     losses.append(loss)
24
25     return thetas_path, losses
```

Hình 9 là kết quả sau khi thực thi đoạn code sau:

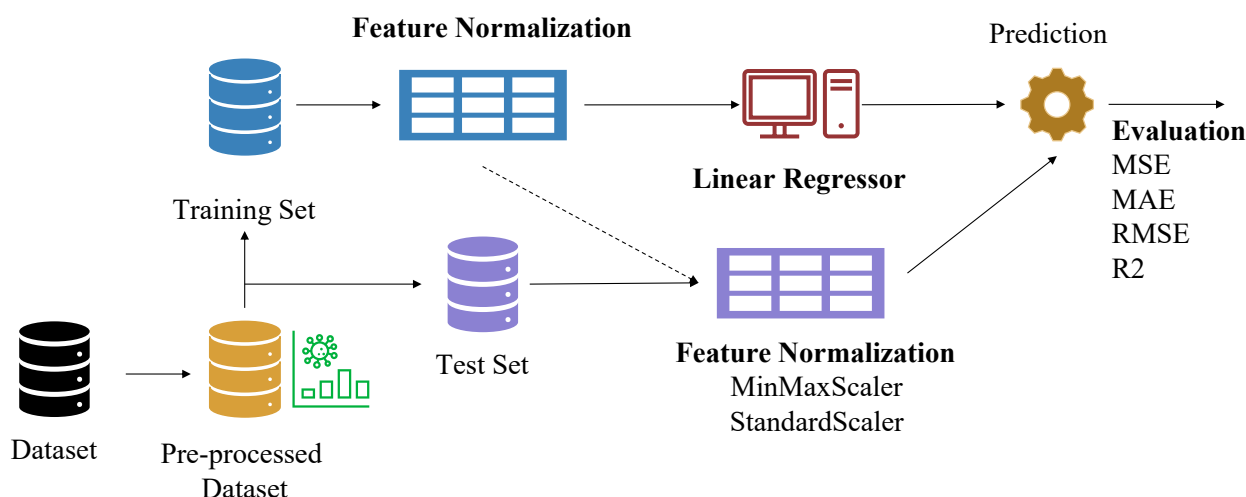
```
1 bgd_thetas, losses = batch_gradient_descent(X_b, y, n_epochs=100, learning_rate=0.01)
2
3 # visualize
4 x_axis = list(range(100))
5 plt.plot(x_axis, losses[:100], color="r")
6 plt.xlabel("Epoch")
7 plt.ylabel("Loss (MSE)")
8 plt.title("Batch Gradient Descent Loss")
9 plt.show()
```



Hình 9: Kết quả loss values sử dụng Batch Gradient Descent.

8. Xây dựng mô hình Linear Regression cho bài toán dự đoán Bitcoin

Ở phần trên, chúng ta áp dụng Linear Regression cho dữ liệu quảng cáo, nơi các mẫu độc lập với nhau. Sang phần này, chúng ta chuyển qua dữ liệu chuỗi thời gian, với bài toán dự đoán giá đóng cửa của Bitcoin từ giá mở cửa, giá cao nhất và giá thấp nhất trong cùng ngày.



Hình 10: Các bước thực hiện xây dựng mô hình hồi quy tuyến tính cho bộ dữ liệu bitcoin.

8.1. Bước 1: Tải và đọc dữ liệu

Đầu tiên, ta tiến hành tải bộ dữ liệu Bitcoin forecasting bằng code sau:

```
1 !gdown 10saJePZ-cwkc0mmiN1V1wgtWLMiJW9VA
2 !unzip BTC-Daily.zip
```

Import các thư viện cần thiết và tiến hành đọc file.csv như sau:

```
1 import pandas as pd
2 import numpy as np
3 import datetime as dt
4 import matplotlib.pyplot as plt
5
6 df = pd.read_csv("../content/BTC-Daily.csv")
7 df = df.drop_duplicates()
8 df.head()
```

Ở bước này ta sử dụng `pandas` để đọc dữ liệu lịch sử giá Bitcoin từ file `BTC-Daily.csv`. Lệnh `drop_duplicates()` giúp loại bỏ các dòng trùng lặp, đảm bảo dữ liệu sạch. Cuối cùng,

`df.head()` hiển thị 5 dòng đầu tiên của bảng dữ liệu, cho phép ta kiểm tra nhanh cấu trúc và các cột trong dataset.

	unix	date	symbol	open	high	low	close	Volume BTC	Volume USD
0	1646092800	2022-03-01 00:00:00	BTC/USD	43221.71	43626.49	43185.48	43185.48	49.006289	2.116360e+06
1	1646006400	2022-02-28 00:00:00	BTC/USD	37717.10	44256.08	37468.99	43178.98	3160.618070	1.364723e+08
2	1645920000	2022-02-27 00:00:00	BTC/USD	39146.66	39886.92	37015.74	37712.68	1701.817043	6.418008e+07
3	1645833600	2022-02-26 00:00:00	BTC/USD	39242.64	40330.99	38600.00	39146.66	912.724087	3.573010e+07
4	1645747200	2022-02-25 00:00:00	BTC/USD	38360.93	39727.97	38027.61	39231.64	2202.851827	8.642149e+07

Hình 11: Vài mẫu dữ liệu của dataset Bitcoin forecasting.

8.2. Bước 2: Khám phá dữ liệu

```

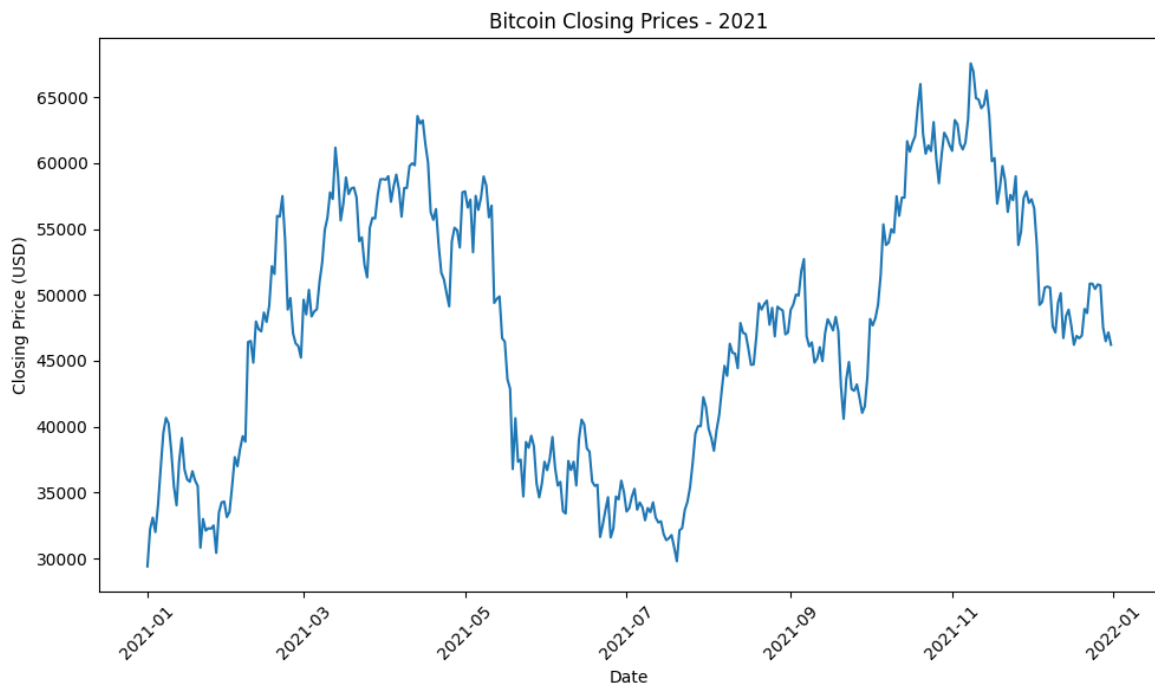
1 # Range of dates covered
2 df["date"] = pd.to_datetime(df["date"])
3 date_range = str(df["date"].dt.date.min()) + " to " + str(df["date"].dt.date.max())
4 print(date_range)
5
6 df["year"] = df["date"].dt.year
7 df["month"] = df["date"].dt.month
8 df["day"] = df["date"].dt.day
9
10
11 unique_years = df["year"].unique()
12 for year in unique_years:
13
14     dates = pd.date_range(start=f"{year}-01-01", end=f"{year}-12-31", freq="D")
15     year_month_day = pd.DataFrame({"date": dates})
16     year_month_day["year"] = year_month_day["date"].dt.year
17     year_month_day["month"] = year_month_day["date"].dt.month
18     year_month_day["day"] = year_month_day["date"].dt.day
19
20
21 merged_data = pd.merge(year_month_day, df, on=["year", "month", "day"], how="left")
22
23 # Plot
24 plt.figure(figsize=(10, 6))
25 plt.plot(merged_data["date_x"], merged_data["close"])
26 plt.title(f"Bitcoin Closing Prices - {year}")
27 plt.xlabel("Date")
28 plt.ylabel("Closing Price (USD)")
29 plt.xticks(rotation=45)
30 plt.tight_layout()
31 plt.show()

```

Trong bước này, dữ liệu được xử lý và trực quan hoá theo các bước:

- Chuyển đổi dữ liệu về dạng ngày tháng để xác định khoảng thời gian mà tập dữ liệu bao phủ.

- Tách thêm các cột `year`, `month`, `day` nhằm dễ dàng thao tác theo từng năm.
- Với mỗi năm, dữ liệu được ghép với dãy ngày đầy đủ để tránh bị thiếu ngày (nếu thị trường nghỉ giao dịch hoặc dữ liệu không đủ).
- Vẽ biểu đồ giá đóng cửa (`close`) theo thời gian cho từng năm, giúp quan sát trực quan biến động giá Bitcoin và phát hiện các xu hướng chính trong từng giai đoạn.



Hình 12: Biểu đồ giá kết thúc phiên trong năm 2021.

8.3. Bước 3: Tạo biểu đồ nền giá giao dịch từ năm 2019 - 2022

Cài đặt thư viện `mplfinance` để vẽ biểu đồ nền giao dịch:

```
1 !pip install mplfinance
```

Tiếp theo, ta thực thi code bên dưới để tạo biểu đồ giao dịch từ năm 2019 - 2022:

```
1 import matplotlib.pyplot as plt
2 import matplotlib.dates as mdates
3 from mplfinance.original_flavor import candlestick_ohlc
4 import datetime
5
6 # Filter data for 2019-2022, convert date to matplotlib format
7 df_filtered = df[(df["date"] >= "2019-01-01") & (df["date"] <= "2022-12-31")].copy()
8 df_filtered["date"] = df_filtered["date"].map(mdates.date2num)
9
10
11 # Create the candlestick chart
```

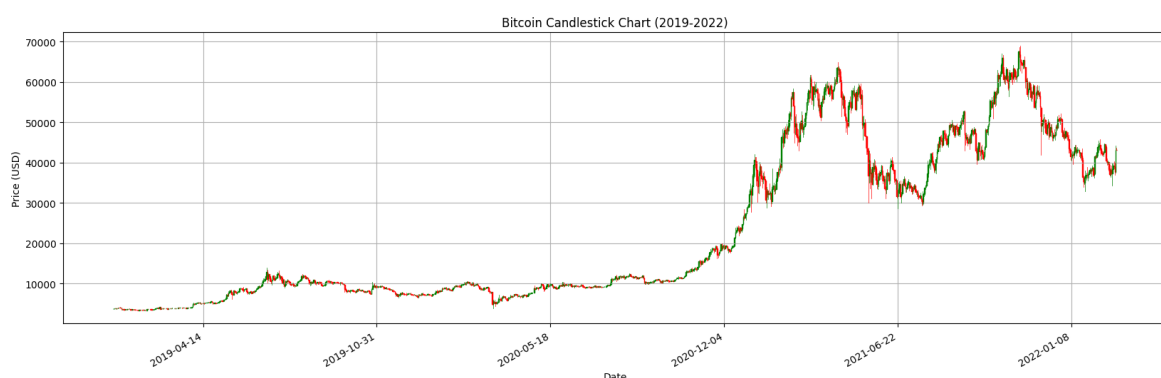
```

12 fig, ax = plt.subplots(figsize=(20, 6))
13
14 candlestick_ohlc(ax, df_filtered[["date", "open", "high", "low", "close"]].values,
15                        width=0.6, colorup="g", colordown="r")
16
17 ax.xaxis.set_major_formatter(mdates.DateFormatter("%Y-%m-%d"))
18 fig.autofmt_xdate()
19
20 plt.title("Bitcoin Candlestick Chart (2019-2022)")
21 plt.xlabel("Date")
22 plt.ylabel("Price (USD)")
23 plt.grid(True)
24
25 # Save the plot as a PDF
26 plt.savefig("bitcoin_candlestick_2019_2022.pdf")
27
28 plt.show()

```

Trong bước này, dữ liệu được lọc cho giai đoạn từ năm 2019 đến 2022 và biểu diễn dưới dạng biểu đồ nến (candlestick).

- Mỗi nến thể hiện biến động giá trong một ngày giao dịch, gồm 4 giá trị **open**, **high**, **low** và **close**.
- Nến màu xanh (green) biểu thị giá đóng cửa cao hơn giá mở cửa, trong khi nến màu đỏ (red) thể hiện chiều ngược lại.
- Biểu đồ nến cho phép quan sát trực quan xu hướng tăng/giảm trong ngắn hạn và các biến động mạnh của giá Bitcoin trong từng giai đoạn.
- Kết quả cuối cùng được lưu ra file PDF để thuận tiện cho việc sử dụng trong báo cáo.



Hình 13: Biểu đồ giao dịch từ năm 2019 - 2022.

8.4. Bước 4: Tạo mô hình Linear Regression

☰ **Yêu cầu 9:** Hoàn thành ba hàm **predict**, **gradient** và **update_weight** làm nền cho quá trình huấn luyện.

```

1 def predict(X, w, b):
2     ***Your code here***
3
4 def gradient(y_hat, y, x):
5     ***Your code here***
6     return (dw,db, cost)
7
8 def update_weight(w,b,lr,dw,db):
9     ***Your code here***
10    return (w_new, b_new)

```

8.5. Bước 5: Chia tập train và test

```

1 from sklearn.preprocessing import StandardScaler
2 from sklearn.model_selection import train_test_split
3
4 X = df[["open", "low", "high"]].values
5 y = df["close"].values
6
7 X_train, X_test, y_train, y_test = train_test_split(
8     X, y, test_size=0.3, random_state=42, shuffle=True
9 )
10
11 scaler = StandardScaler()
12
13 X_train_scaled = scaler.fit_transform(X_train)
14 X_test_scaled = scaler.transform(X_test)

```

Tách dữ liệu:

- X : các đặc trưng open, low, high.
- y : nhãn close.

Chia dữ liệu bằng `train_test_split` (70% train, 30% test, `random_state=42`).

Chuẩn hoá dữ liệu với `StandardScaler`:

- `fit_transform` trên tập train.
- `transform` trên tập test.

8.6. Bước 6: Huấn luyện và đánh giá mô hình tự xây dựng

Yêu cầu 10: Hoàn thành hàm `linear_regression_vectorized()` để huấn luyện mô hình bằng Batch Gradient Descent.

```

1 def linear_regression_vectorized(X, y, learning_rate=0.01, num_iterations=200):
2     n_samples, n_features = X.shape

```

```

3  w = np.zeros(n_features) # Initialize weights
4  b = 0 # Initialize bias
5  losses = []
6
7  ***Your code here***
8
9  return w, b, losses

```

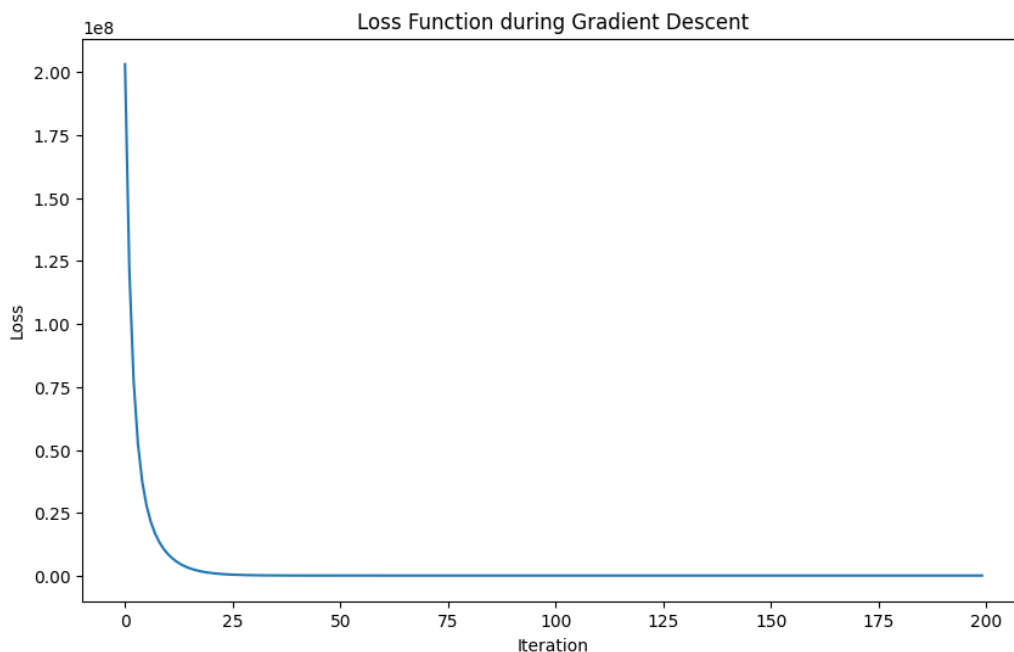
Huấn luyện mô hình bằng Batch Gradient Descent và trực quan hoá quá trình tối ưu bằng cách vẽ biểu đồ hàm loss theo số vòng lặp để quan sát mức độ hội tụ của mô hình.

Để chọn learning rate, chúng ta dò trên 50 giá trị trải đều trong khoảng $[0.0001, 0.1]$ và đánh giá từng giá trị bằng cross-validation 5-fold trên tập train, mỗi lần huấn luyện 500 vòng lặp. Từ khoảng 0.01 trở lên, R^2 trung bình gần như không đổi và đạt khoảng 0.9989. Giá trị cao nhất xuất hiện ở $\eta = 0.1$, cũng là cận trên của khoảng dò, nên ta dùng $\eta = 0.1$ cho bước huấn luyện dưới đây. Phần code thực hiện bước dò này nằm trong notebook đi kèm.

```

1  w, b, losses = linear_regression_vectorized(X_train_scaled, y_train, learning_rate=0.1
2                                          , num_iterations=200)
3  # Plot the loss function
4  plt.figure(figsize=(10, 6))
5  plt.plot(losses)
6  plt.xlabel("Iteration")
7  plt.ylabel("Loss")
8  plt.title("Loss Function during Gradient Descent")
9  plt.show()

```



Hình 14: Biểu đồ hàm loss trong quá trình huấn luyện.

Sau khi mô hình được huấn luyện, ta tiến hành đánh giá hiệu quả dự đoán bằng cách tính các chỉ số sai số phổ biến gồm **RMSE**, **MAE** và **MAPE** trên tập kiểm tra, cùng hệ số xác định R^2 trên cả tập huấn luyện và tập kiểm tra. Các chỉ số này giúp kiểm chứng độ chính xác và khả năng khái quát hoá của mô hình.

```

1 from sklearn.metrics import r2_score
2
3 # Make predictions on the test set
4 y_pred = predict(X_test_scaled, w, b)
5
6 # Calculate RMSE
7 rmse = np.sqrt(np.mean((y_pred - y_test) ** 2))
8
9 # Calculate MAE
10 mae = np.mean(np.abs(y_pred - y_test))
11
12 # Calculate MAPE
13 mape = np.mean(np.abs((y_test - y_pred) / y_test)) * 100
14
15
16 # Calculate R-squared on training data
17 y_train_pred = predict(X_train_scaled, w, b)
18 train_accuracy = r2_score(y_train, y_train_pred)
19
20 # Calculate R-squared on testing data
21 test_accuracy = r2_score(y_test, y_pred)
22
23 print("Root Mean Square Error (RMSE):", round(rmse, 4))
24 print("Mean Absolute Error (MAE):", round(mae, 4))
25 print("Mean Absolute Percentage Error (MAPE):", round(mape, 4))
26 print("Training Accuracy (R-squared):", round(train_accuracy, 4))
27 print("Testing Accuracy (R-squared):", round(test_accuracy, 4))

```

Để minh hoạ kết quả dự đoán, ta thử nghiệm trên dữ liệu quý 1 năm 2019 (từ 01/01 đến 31/03). Biểu đồ dưới đây thể hiện sự so sánh giữa **giá thực tế** và **giá dự đoán**, giúp quan sát trực quan mức độ sai lệch và xu hướng giữa hai đường giá.

```

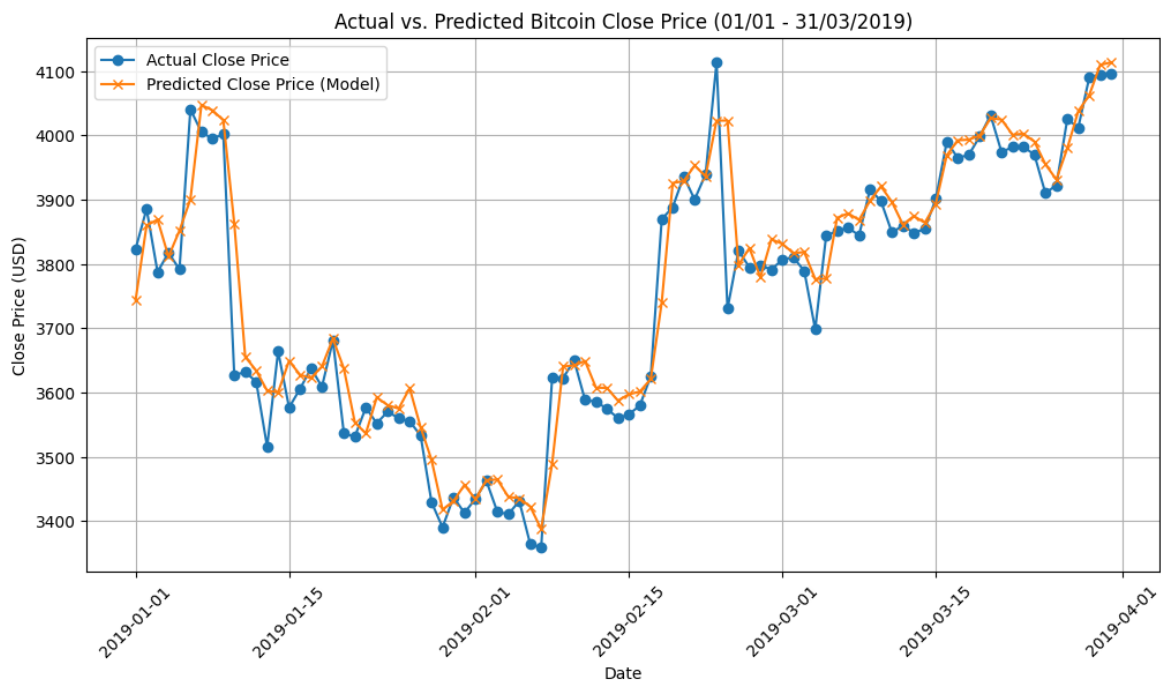
1 # Lọc dữ liệu 2019 Q1
2 df_2019_Q1 = df[(df["date"] >= "2019-01-01") & (df["date"] <= "2019-03-31")].copy()
3
4 # Chọn đúng các cột feature đã train
5 X_2019 = df_2019_Q1[["open", "low", "high"]].values
6 X_2019_scaled = scaler.transform(X_2019) # dùng scaler đã fit trước đó
7 y_2019 = df_2019_Q1["close"].values
8
9 # Dự đoán bằng mô hình
10 y_pred_2019 = predict(X_2019_scaled, w, b)
11
12 # Gắn vào dataframe để dễ vẽ
13 df_2019_Q1["predicted_close"] = y_pred_2019
14
15 # Vẽ biểu đồ
16 plt.figure(figsize=(12,6))

```

```

17 plt.plot(df_2019_Q1["date"], df_2019_Q1["close"], label="Actual Close Price", marker="o")
18 plt.plot(df_2019_Q1["date"], df_2019_Q1["predicted_close"], label="Predicted Close Price (Model)", marker="x")
19 plt.title("Actual vs. Predicted Bitcoin Close Price (01/01 - 31/03/2019)")
20 plt.xlabel("Date")
21 plt.ylabel("Close Price (USD)")
22 plt.xticks(rotation=45)
23 plt.legend()
24 plt.grid(True)
25 plt.show()

```



Hình 15: Biểu đồ so sánh giá kết thúc phiên thực tế và dự đoán của Bitcoin (01/01/2019 - 31/03/2019).

8.7. Bước 7: Huấn luyện và đánh giá mô hình từ thư viện scikit-learn

```

1 from sklearn.linear_model import LinearRegression
2
3 linear_regressor = LinearRegression()
4 linear_regressor.fit(X_train_scaled, y_train)

```

Mô hình được xây dựng bằng `LinearRegression` của `scikit-learn`, được huấn luyện trên dữ liệu đã chuẩn hoá để tự động tìm ra các hệ số w và bias b tối ưu.

```

1 from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
2

```

```

3 # Make predictions on the test set
4 y_pred = linear_regressor.predict(X_test_scaled)
5
6 # Evaluation
7 mse = mean_squared_error(y_test, y_pred)
8 mae = mean_absolute_error(y_test, y_pred)
9 r2 = r2_score(y_test, y_pred)
10
11 print("Mean Square Error (MSE):", round(mse, 4))
12 print("Mean Absolute Error (MAE):", round(mae, 4))
13 print("R-squared:", round(r2, 4))

```

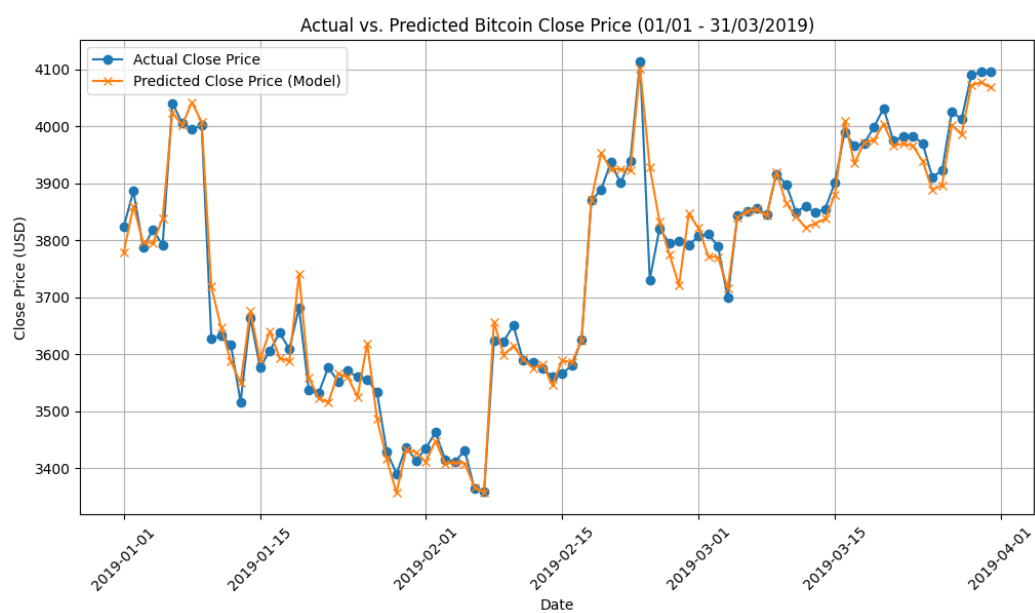
Mô hình sau khi huấn luyện được dùng để dự đoán trên tập kiểm tra, các chỉ số đánh giá gồm MSE (Mean Squared Error), MAE (Mean Absolute Error) và R^2 (R-squared) được tính toán để đo lường chất lượng dự báo.

```

1 # Lọc dữ liệu 2019 Q1
2 df_2019_Q1 = df[(df["date"] >= "2019-01-01") & (df["date"] <= "2019-03-31")].copy()
3
4 # Chọn đúng các cột feature đã train
5 X_2019 = df_2019_Q1[["open", "low", "high"]].values
6 X_2019_scaled = scaler.transform(X_2019)
7 y_2019 = df_2019_Q1["close"].values
8
9 # Dự đoán bằng model sklearn
10 y_pred_2019 = linear_regressor.predict(X_2019_scaled)
11
12 # Gắn vào dataframe để tiện vẽ
13 df_2019_Q1["predicted_close"] = y_pred_2019
14
15 # Vẽ
16 plt.figure(figsize=(12,6))
17 plt.plot(df_2019_Q1["date"], df_2019_Q1["close"], label="Actual Close Price", marker="o")
18 plt.plot(df_2019_Q1["date"], df_2019_Q1["predicted_close"], label="Predicted Close Price (Model)", marker="x")
19 plt.title("Actual vs. Predicted Bitcoin Close Price (01/01 - 31/03/2019)")
20 plt.xlabel("Date")
21 plt.ylabel("Close Price (USD)")
22 plt.xticks(rotation=45)
23 plt.legend()
24 plt.grid(True)
25 plt.show()

```

Dữ liệu được lọc cho giai đoạn Q1/2019, sau đó chuẩn hoá bằng scaler đã huấn luyện. Mô hình LinearRegression được sử dụng để dự đoán giá đóng cửa, và kết quả được so sánh trực quan với giá thực tế thông qua biểu đồ đường.



Hình 16: Biểu đồ so sánh giá kết thúc phiên thực tế và dự đoán của Bitcoin (dùng sklearn) (01/01/2019 - 31/03/2019).

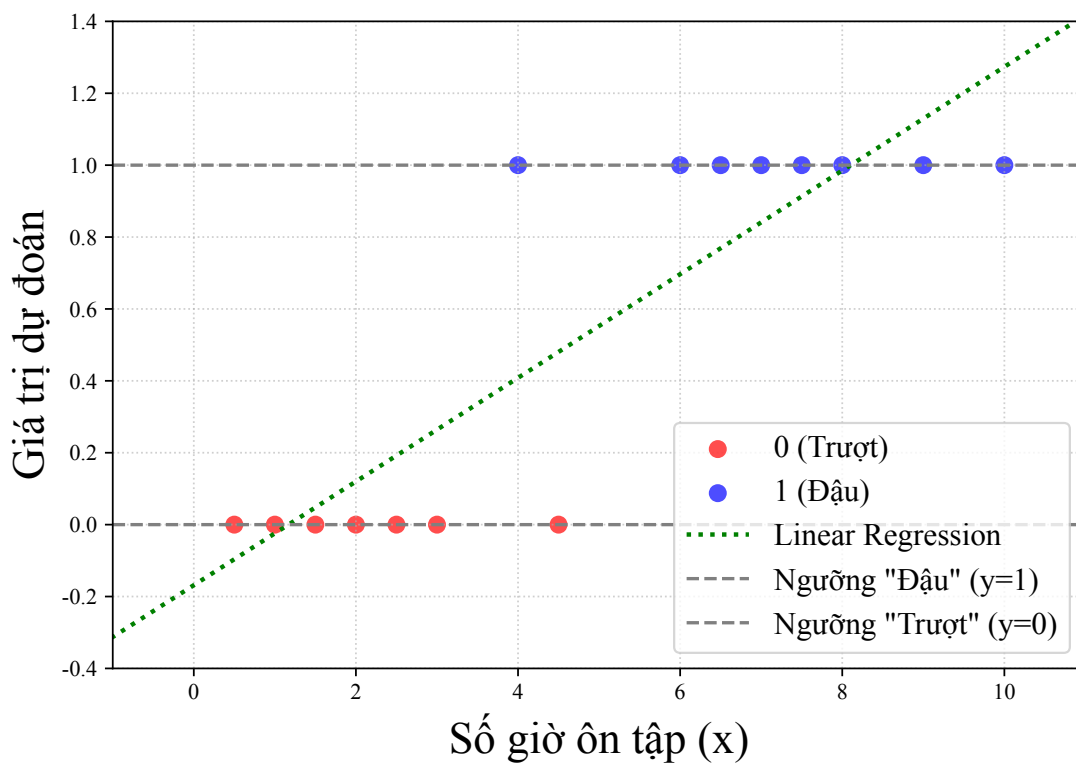
9. Từ Linear Regression đến Logistic Regression

9.1. Bài toán phân loại nhị phân

Ở các phần trên, chúng ta đã tìm hiểu cách dự đoán một giá trị liên tục như doanh số hay giá Bitcoin. Đường thẳng $f(x) = wx + b$ hoạt động tốt vì đầu ra y có thể nhận bất kỳ giá trị số thực nào. Tuy nhiên, trong thực tế, có nhiều bài toán mà đầu ra không phải là một con số liên tục, mà là một sự lựa chọn giữa hai hoặc nhiều khả năng. Ví dụ:

- Dự đoán một email là spam hay không spam? (0 hoặc 1)
- Dự đoán một khối u là lành tính hay ác tính? (0 hoặc 1)
- Dự đoán một sinh viên thi Đậu hay thi trượt dựa trên số giờ ôn tập? (1 hoặc 0)

Đây được gọi là các bài toán phân loại nhị phân hay Binary Classification.



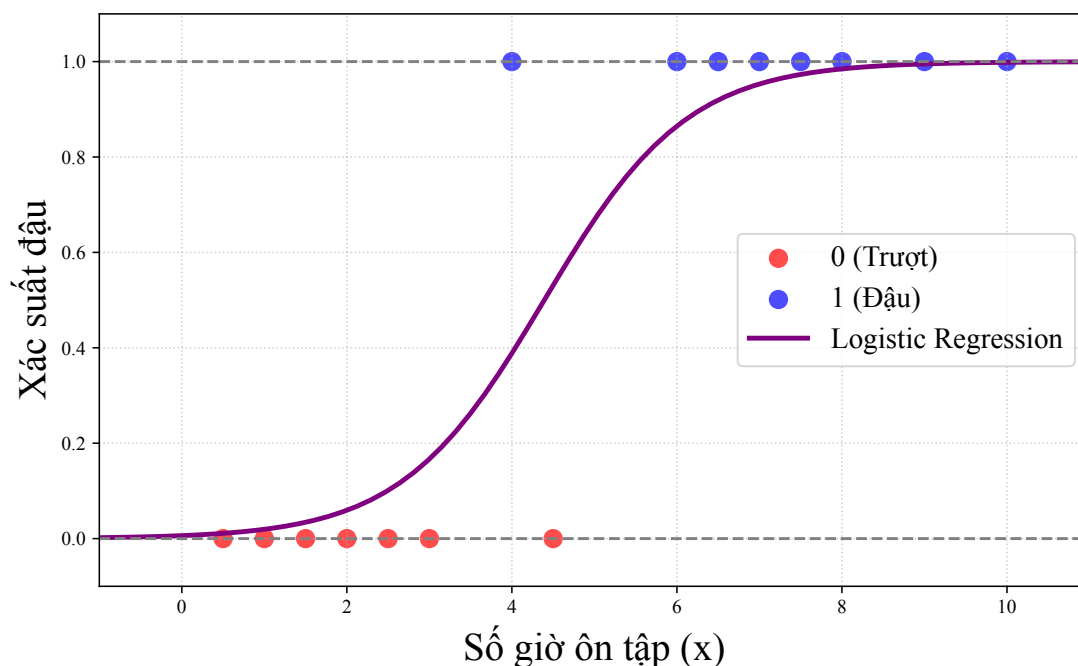
Hình 17: Đường thẳng Linear Regression dự đoán vượt ra ngoài các giá trị có ý nghĩa $[0, 1]$.

Hãy xét ví dụ về sinh viên. Trục hoành x là số giờ ôn tập, còn trục tung y chỉ nhận hai giá trị 0 (trượt) và 1 (Đậu). Nếu ta cố gắng dùng Linear Regression để khớp một đường thẳng $f(x) = wx + b$ với dữ liệu này, ta sẽ gặp các vấn đề:

1. Đường thẳng có thể dự đoán các giá trị lớn hơn 1 (ví dụ: $f(x) = 1.5$, ứng với xác suất Đậu bằng 150%) hoặc nhỏ hơn 0 (ví dụ: $f(x) = -0.3$, ứng với xác suất Đậu bằng -30%). Những con số này dường như không có ý nghĩa thực tế với bài toán.
2. Chúng ta mong muốn đầu ra là một xác suất, ví dụ như $P(\text{Đậu}|x \text{ giờ học})$, và xác suất về bản chất phải nằm trong đoạn $[0, 1]$. Tuy nhiên, đầu ra của Linear Regression là một số

thực bất kỳ, có thể vượt ra ngoài đoạn này (như 1.5 hoặc -0.3), do đó nó không đáp ứng được mong muốn này.

Để giải quyết vấn đề này, chúng ta cần một mô hình mới có khả năng nhận đầu ra của hàm tuyến tính (vốn là một số thực bất kỳ từ $-\infty$ đến $+\infty$) và biến đổi nó lại một cách mượt mà vào khoảng $(0, 1)$.



Hình 18: Logistic Regression (màu tím) sử dụng hàm Sigmoid để biến đổi kết quả dự đoán thành một xác suất, luôn nằm trong khoảng $(0, 1)$.

Logistic Regression đáp ứng yêu cầu này. Mô hình đưa đầu ra tuyến tính qua hàm Sigmoid để thu được một giá trị trong khoảng $(0, 1)$, diễn giải được thành xác suất. Các mục tiếp theo trình bày hàm Sigmoid, hàm loss dành cho bài toán phân loại, cách tính đạo hàm của hàm loss và quy trình huấn luyện mô hình.

9.2. Thuật ngữ và ký hiệu

Hai bảng dưới đây liệt kê các thuật ngữ và ký hiệu dùng trong phần này.

Bảng thuật ngữ

Thuật ngữ	Ý nghĩa
Phân loại nhị phân	Một bài toán học máy nhằm dự đoán đầu ra thuộc về một trong hai nhóm, thường được gán nhãn là 0 hoặc 1.
Logistic Regression	Mô hình học máy dùng cho bài toán phân loại nhị phân. Mô hình này dự đoán xác suất một đối tượng thuộc về lớp 1.
Hàm Sigmoid (Hàm Logistic)	Một hàm số có đồ thị hình chữ S dùng để ánh xạ một số thực bất kỳ, ví dụ như đầu ra của Linear Regression, vào khoảng từ 0 đến 1.
Loss Function	Một hàm số đo sai lệch giữa giá trị dự đoán của mô hình và giá trị thực tế trên một mẫu dữ liệu.
Binary Cross-Entropy (BCE)	Hàm loss tiêu chuẩn dùng cho Logistic Regression. Giá trị của hàm tăng nhanh khi xác suất dự đoán lệch xa nhãn thực tế và tiến về 0 khi dự đoán đúng.
Hàm lồi (Convex)	Một hàm số mà đoạn thẳng nối hai điểm bất kỳ trên đồ thị luôn nằm trên hoặc trùng với đồ thị. Hàm loss BCE là hàm lồi, nhờ đó mọi điểm dừng của Gradient Descent đều là cực tiểu toàn cục.
Hàm không lồi (Non-convex)	Một hàm số không thỏa điều kiện lồi, nên có thể có nhiều cực tiểu địa phương. Dùng MSE cho Logistic Regression sẽ tạo ra hàm này, khiến thuật toán tối ưu có thể bị kẹt.
Gradient Descent	Thuật toán tối ưu dùng để tìm giá trị tham số làm cho hàm loss nhỏ nhất, bằng cách đi ngược hướng đạo hàm.
Batch Gradient Descent	Một phiên bản của Gradient Descent, trong đó đạo hàm được tính trung bình trên toàn bộ mẫu dữ liệu trước khi cập nhật.
Siêu tham số	Các giá trị được con người thiết lập trước khi huấn luyện, ví dụ như learning rate hay số Epoch, để kiểm soát quá trình huấn luyện.
Learning Rate	Một siêu tham số, ký hiệu là η , quyết định độ lớn của mỗi bước cập nhật tham số trong mỗi vòng lặp Gradient Descent.
Epoch	Một epoch có nghĩa là mô hình đã học hỏi từ toàn bộ tất cả m mẫu dữ liệu trong tập huấn luyện đúng một lần.

Bảng Ký hiệu Toán học

Ký hiệu	Ý nghĩa
m	Số mẫu dùng cho một lần cập nhật tham số. Trong phần này, m bằng kích thước toàn tập huấn luyện (full-batch).
x	Biến đầu vào, hay đặc trưng của dữ liệu.
y	Biến đầu ra, hay nhãn thực tế.
(x_i, y_i)	Mẫu dữ liệu thứ i trong tập huấn luyện. x_i là đầu vào và y_i là nhãn thực tế của mẫu đó.
w	Trọng số của mô hình. Đây là tham số mô hình học được, cho biết x ảnh hưởng nhiều hay ít và tích cực hay tiêu cực đến kết quả.
b	Bias của mô hình. Đây là tham số mô hình học được, giúp điều chỉnh đường biên quyết định.
z_i	Đầu vào tuyến tính, còn gọi là logit. Đây là kết quả của phép tính $z_i = wx_i + b$ trước khi đưa vào hàm Sigmoid.
$g(z)$	Hàm Sigmoid, hay hàm Logistic. Đây là hàm $g(z) = \frac{1}{1+e^{-z}}$ dùng để biến z thành xác suất.
$\hat{y}_i$	Xác suất dự đoán. Đây là đầu ra của mô hình sau khi đưa z_i qua hàm Sigmoid, $\hat{y}_i = g(z_i)$. Nó biểu thị xác suất $y_i = 1$.
$P(y = 1 x)$	Ký hiệu toán học của xác suất có điều kiện, đọc là “xác suất $y = 1$ khi biết đầu vào là x ”. Đây chính là giá trị mà $\hat{y}_i$ cố gắng dự đoán.
L	Hàm loss Binary Cross-Entropy, đo lường lỗi của dự đoán $\hat{y}$ so với nhãn y .
$\log(\cdot)$	Hàm Logarit tự nhiên. Đây là thành phần chính trong hàm loss Binary Cross-Entropy, làm loss tăng rất nhanh khi dự đoán lệch xa nhãn thực tế.
$\frac{\partial L}{\partial w}$	Đạo hàm riêng của hàm L theo trọng số w . Nó cho biết L thay đổi nhanh thế nào khi w thay đổi, và là la bàn cho Gradient Descent.
$\frac{\partial L}{\partial b}$	Đạo hàm riêng của hàm L theo b . Nó cho biết L thay đổi nhanh thế nào khi b thay đổi.
η	Learning rate, đọc là “eta”. Đây là một siêu tham số, quyết định độ lớn của mỗi bước cập nhật w và b trong Gradient Descent.

9.3. Ôn tập nhanh Linear Regression

Trước khi chuyển sang bài toán phân loại, ta tóm tắt lại các thành phần đã xây dựng ở phần Linear Regression:

1. **Phương trình dự đoán:** Chúng ta dùng một hàm tuyến tính, về cơ bản là một đường

thẳng, để mô hình hoá mối quan hệ và dự đoán giá trị $\hat{y}$:

$$f(x) = \hat{y} = wx + b.$$

2. **Hàm Loss:** Để đánh giá mức độ chính xác của mô hình, chúng ta cần một hàm lỗi $L(w, b)$. Mục tiêu là tìm w và b sao cho hàm này đạt giá trị nhỏ nhất. Trong Linear Regression, chúng ta dùng Mean Squared Error (MSE):

$$L(w, b) = \frac{1}{m} \sum_{i=1}^m (\hat{y}_i - y_i)^2 = \frac{1}{m} \sum_{i=1}^m (wx_i + b - y_i)^2.$$

Hàm này tính trung bình của bình phương chênh lệch giữa giá trị dự đoán và giá trị thực tế trên toàn bộ m mẫu dữ liệu.

3. **Tối ưu hoá:** Sau khi có hàm loss, chúng ta sử dụng thuật toán Gradient Descent để tìm w và b tối ưu. Thuật toán này cập nhật w và b một cách lặp đi lặp lại bằng cách đi ngược hướng đạo hàm (độ dốc) của hàm loss:

$$w = w - \eta \frac{\partial L}{\partial w},$$

$$b = b - \eta \frac{\partial L}{\partial b}.$$

Vấn đề còn tồn tại: Linear Regression hoạt động tốt cho việc dự đoán các giá trị liên tục. Tuy nhiên, khi áp dụng vào bài toán phân loại, đầu ra $\hat{y} = wx + b$ của nó là một số thực bất kỳ (từ $-\infty$ đến $+\infty$). Giá trị này không thể được diễn giải trực tiếp là một xác suất (vốn dĩ phải nằm trong đoạn $[0, 1]$). Đây chính là điểm mấu chốt. Chúng ta cần một cách để biến đổi giá trị $wx + b$ này vào khoảng $(0, 1)$.

9.4. Hàm Sigmoid

Như đã phân tích, chúng ta cần một hàm số nhận đầu vào là $z = wx + b$ (một số thực bất kỳ từ $-\infty$ đến $+\infty$) và ánh xạ nó một cách mượt mà vào khoảng $(0, 1)$ để có thể diễn giải là một xác suất.

i Có thể bạn chưa biết: “Sigmoid” vs. “Logistic”

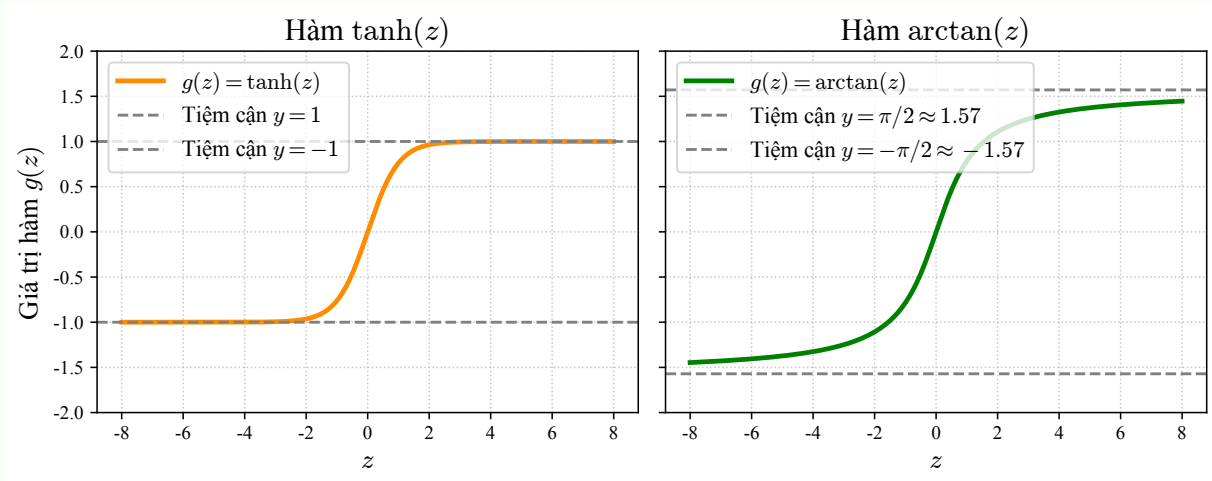
Hai thuật ngữ này thường bị dùng lẫn với nhau. Phân biệt như sau:

- **Hàm Sigmoid:** Đây là tên gọi chung cho cả một họ các hàm số có chung đặc điểm là đồ thị hình chữ “S” (S-shaped). Chúng nhận mọi số thực làm đầu vào, tăng đơn điệu, cho đầu ra bị chặn trong một khoảng nhất định và có đúng một điểm uốn.
- **Hàm Logistic:** Đây là một thành viên cụ thể và cũng là thành viên nổi tiếng nhất trong họ Sigmoid. Dạng chuẩn của nó là $g(z) = \frac{1}{1+e^{-z}}$, ánh xạ đầu ra vào khoảng $(0, 1)$.

Trong học máy, đặc biệt là trong mô hình Logistic Regression, hàm Logistic được dùng phổ biến đến mức người ta thường gọi nó là “hàm Sigmoid”.

Một số ví dụ khác về các hàm cũng thuộc họ Sigmoid bao gồm:

- **Hàm Hyperbolic Tangent** ($\tanh$): Có công thức $\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$. Nó cũng có hình chữ S nhưng có giá trị đầu ra trong khoảng $(-1, 1)$.
- **Hàm Arctangent** ($\arctan$): Có giá trị đầu ra trong khoảng $(-\pi/2, \pi/2)$.



Hình 19: Hai hàm khác thuộc họ Sigmoid. (Trái) $\tanh(z)$ có tiệm cận $y = \pm 1$. (Phải) $\arctan(z)$ có tiệm cận $y = \pm\pi/2 \approx \pm 1.57$. Cả hai đều có dạng chữ S nhưng khoảng giá trị đầu ra khác với hàm Logistic.

Tóm lại: Trong bài học này, khi chúng ta nói “Hàm Sigmoid”, chúng ta đang đề cập đến “Hàm Logistic” $g(z) = \frac{1}{1+e^{-z}}$.

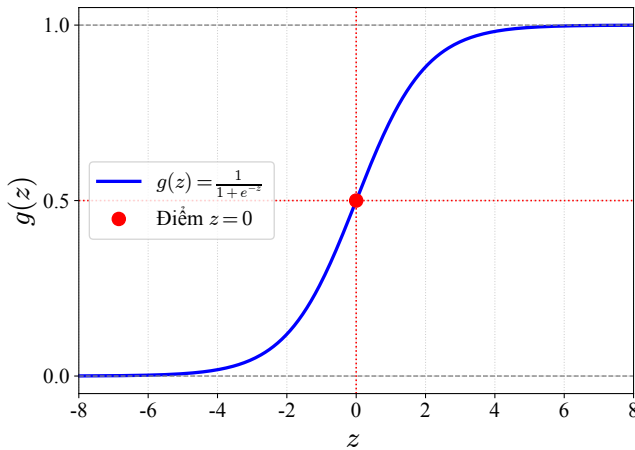
Khi đã làm rõ điều đó, chúng ta sẽ sử dụng hàm Logistic cho mô hình của mình, có công thức như sau:

$$g(z) = \frac{1}{1 + e^{-z}}.$$

Trong đó, $z = wx + b$ chính là đầu ra của hàm tuyến tính mà chúng ta đã quen thuộc. Khi kết hợp lại, phương trình dự đoán $\hat{y}$ của Logistic Regression có dạng:

$$\hat{y} = g(wx + b) = \frac{1}{1 + e^{-(wx+b)}}.$$

Để hiểu tại sao hàm này lại hữu ích, chúng ta hãy cùng xem đồ thị và các tính chất toán học của nó:



Hình 20: Đồ thị của hàm $g(z) = \frac{1}{1+e^{-z}}$.

- **Khi z rất lớn ($z \rightarrow +\infty$):** Giá trị e^{-z} sẽ tiến rất nhanh về 0. Do đó, $g(z) \approx \frac{1}{1+0} = 1$.
- **Khi z rất nhỏ ($z \rightarrow -\infty$):** Giá trị e^{-z} sẽ trở nên cực kỳ lớn (tiến đến $+\infty$). Khi đó, mẫu số $(1 + e^{-z})$ cũng tiến đến $+\infty$, và $g(z) \approx \frac{1}{+\infty} \approx 0$.
- **Khi $z = 0$:** Ngay tại trung tâm, ta có $e^{-0} = 1$. Do đó, $g(z) = \frac{1}{1+1} = 0.5$.

Như vậy, bất kể z là bao nhiêu, hàm Logistic luôn trả về $\hat{y}$ nằm trong khoảng $(0, 1)$. Giờ đây, chúng ta có thể diễn giải $\hat{y}$ một cách hợp lý. Đó chính là xác suất dự đoán của mô hình rằng mẫu đó thuộc về lớp 1, ký hiệu là $P(y = 1|x)$.

9.4.1. Ngưỡng quyết định

Sau khi mô hình trả về một xác suất $\hat{y}$ (ví dụ: 0.7, nghĩa là 70% khả năng Đậu), làm thế nào để chúng ta đưa ra quyết định cuối cùng là 0 (Trượt) hay 1 (Đậu)?

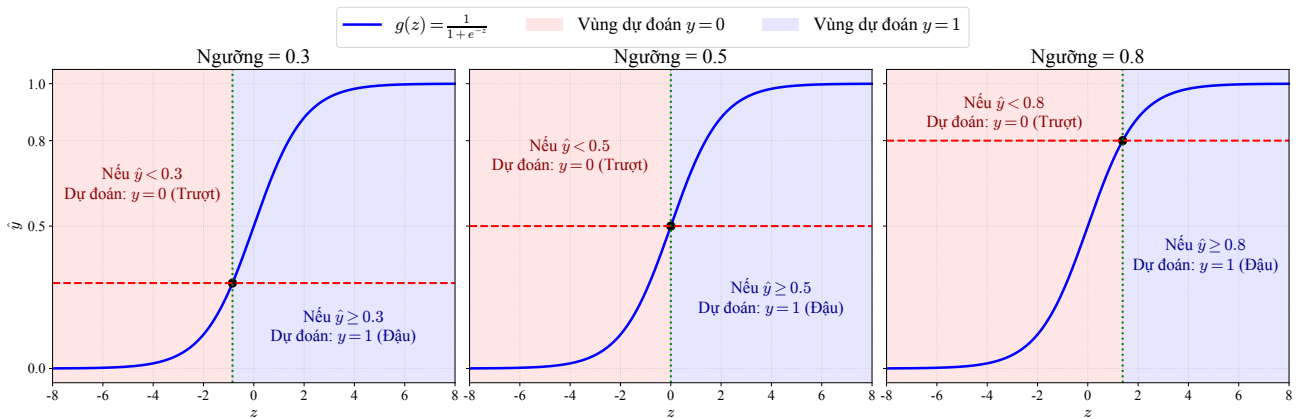
Chúng ta chỉ cần đặt ra một ngưỡng quyết định (threshold). Giá trị thông dụng nhất là 0.5:

- Nếu $\hat{y} \geq 0.5$ (xác suất Đậu từ 50% trở lên), ta dự đoán $y = 1$ (Đậu).
- Nếu $\hat{y} < 0.5$ (xác suất Đậu dưới 50%), ta dự đoán $y = 0$ (Trượt).

Như được minh họa rõ ràng trên Hình 21, dựa trên tính chất của hàm Logistic (Sigmoid), ngưỡng $\hat{y} = 0.5$ này tương ứng chính xác với $z = 0$. Điều này có nghĩa là:

- Nếu $z = wx + b \geq 0$, dự đoán là 1.
- Nếu $z = wx + b < 0$, dự đoán là 0.

Đường ranh giới $wx + b = 0$ này được gọi là đường biên quyết định của mô hình.



Hình 21: Minh họa các ngưỡng quyết định. Đường nét đứt màu đỏ là ngưỡng xác suất. Vùng màu xanh dương dẫn đến dự đoán $y = 1$ (Đầu), và vùng màu đỏ dẫn đến dự đoán $y = 0$ (Trượt).

9.5. Hàm loss mới: Binary Cross-Entropy

Ở phần trước, chúng ta đã có một hàm dự đoán $\hat{y}_i = g(z_i)$ cho ra một xác suất. Bây giờ, chúng ta cần một hàm loss để đo mức độ lệch giữa xác suất dự đoán và nhãn thực tế y_i (là 0 hoặc 1).

Một câu hỏi tự nhiên là “Tại sao ta không dùng Mean Squared Error (MSE) như của Linear Regression?”

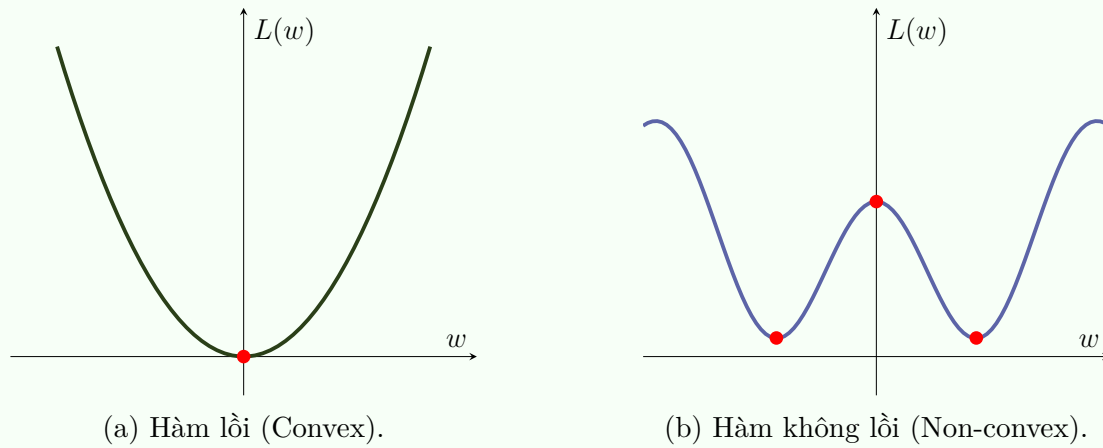
$$\text{MSE: } L(w, b) = \frac{1}{m} \sum_{i=1}^m (\hat{y}_i - y_i)^2 = \frac{1}{m} \sum_{i=1}^m \left(\frac{1}{1 + e^{-(wx_i + b)}} - y_i \right)^2.$$

Lý do là khi kết hợp MSE với hàm Sigmoid, hàm $L(w, b)$ sẽ không còn là một hàm lồi (non-convex) nữa. Đồ thị của nó sẽ có nhiều cực tiểu địa phương. Nếu dùng Gradient Descent, thuật toán có thể dừng lại ở một cực tiểu địa phương mà không tìm được cực tiểu toàn cục, dẫn đến mô hình không tối ưu.

i Lồi (Convex) vs. Không lồi (Non-convex)

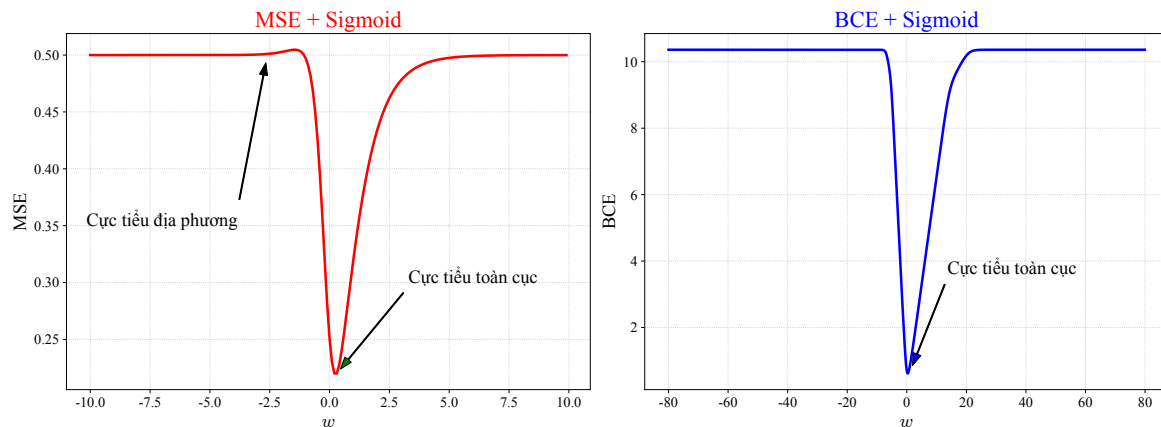
Để hiểu tại sao điều này lại quan trọng, hãy tưởng tượng Gradient Descent là một quả bóng đang lăn xuống dốc, và hàm loss chính là địa hình của con dốc đó:

- **Hàm Lồi (Convex):** Như Hình 22a bên dưới, địa hình của hàm lồi giống như một cái bát trơn. Bất kể bạn thả quả bóng từ vị trí nào, nó sẽ luôn lăn xuống điểm thấp nhất duy nhất ở đáy bát. Điểm này chính là Cực tiểu toàn cục (global minimum).
- **Hàm Không Lồi (Non-convex):** Như Hình 22b, địa hình của hàm không lồi giống như một dãy đồi núi. Nếu bạn thả quả bóng, nó có thể lăn xuống thung lũng gần nhất và bị kẹt ở đó (điểm Cực tiểu địa phương). Quả bóng không thể đi đến một thung lũng khác còn sâu hơn nhiều ở bên kia ngọn đồi (điểm Cực tiểu toàn cục).



Hình 22: So sánh tính lồi của các hàm. (a) Hàm lồi chỉ có một điểm cực tiểu duy nhất. (b) Hàm không lồi có nhiều điểm dừng, nên tùy vào điểm khởi tạo mà Gradient Descent có thể dừng lại ở những nghiệm khác nhau.

Hình 23 minh họa rõ vấn đề này.



Hình 23: So sánh tính convex của MSE và BCE khi kết hợp với hàm Sigmoid. (Trái) MSE + Sigmoid tạo ra hàm loss non-convex. (Phải) BCE + Sigmoid tạo ra hàm loss convex.

Thay vào đó, chúng ta sử dụng một hàm loss được thiết kế riêng cho bài toán phân loại xác suất, đó là **Binary Cross-Entropy (BCE)**, hay còn gọi là **Log Loss**.

9.5.1. Công thức cho một mẫu

Đối với một mẫu dữ liệu (x_i, y_i) , hàm loss L_i được định nghĩa như sau:

$$L_i(\hat{y}_i, y_i) = -[y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)].$$

Công thức này có vẻ phức tạp, nhưng nó thực chất là một cách viết gộp thông minh cho hai trường hợp riêng biệt. Hãy cùng phân tích:

Trường hợp 1: Nhân thực tế $y_i = 1$ (Sinh viên Đậu)

Khi $y_i = 1$, ta thay vào công thức L_i :

$$L_i = -[1 \cdot \log(\hat{y}_i) + (1 - 1) \cdot \log(1 - \hat{y}_i)],$$

$$L_i = -[1 \cdot \log(\hat{y}_i) + 0 \cdot \log(1 - \hat{y}_i)] = -\log(\hat{y}_i).$$

Vậy khi nhãn là 1, chúng ta chỉ cần quan tâm đến $L_i = -\log(\hat{y}_i)$.

- Nếu mô hình dự đoán $\hat{y}_i \rightarrow 1$ (rất gần 1, dự đoán đúng), loss $L_i = -\log(1) \approx 0$. Đây là loss rất nhỏ, vì mô hình đã làm tốt.
- Nếu mô hình dự đoán $\hat{y}_i \rightarrow 0$ (rất gần 0, dự đoán sai), loss $L_i = -\log(0) \rightarrow +\infty$. Đây là loss vô cùng lớn, tương ứng với trường hợp mô hình dự đoán sai hoàn toàn.

Trường hợp 2: Nhãn thực tế $y_i = 0$ (Sinh viên Trượt)

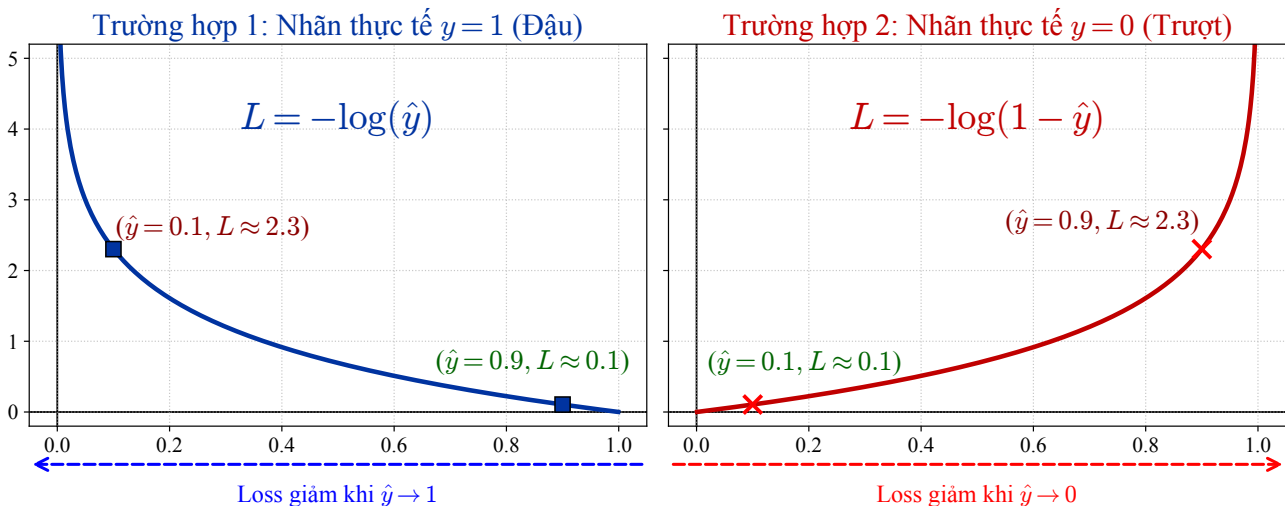
Khi $y_i = 0$, ta thay vào công thức L_i :

$$L_i = -[0 \cdot \log(\hat{y}_i) + (1 - 0) \cdot \log(1 - \hat{y}_i)],$$

$$L_i = -[0 + 1 \cdot \log(1 - \hat{y}_i)] = -\log(1 - \hat{y}_i).$$

Vậy khi nhãn là 0, chúng ta chỉ cần quan tâm đến $L_i = -\log(1 - \hat{y}_i)$.

- Nếu mô hình dự đoán $\hat{y}_i \rightarrow 0$ (rất gần 0, dự đoán đúng), loss $L_i = -\log(1 - 0) \approx -\log(1) \approx 0$. Loss rất nhỏ, mô hình làm tốt.
- Nếu mô hình dự đoán $\hat{y}_i \rightarrow 1$ (rất gần 1, dự đoán sai), loss $L_i = -\log(1 - 1) \rightarrow -\log(0) \rightarrow +\infty$. Loss vô cùng lớn, ứng với dự đoán sai hoàn toàn.



Hình 24: Trực quan hoá hàm loss Binary Cross-Entropy. Khi nhãn thực tế $y = 1$, ta sử dụng hàm loss $L = -\log(\hat{y})$. Khi nhãn thực tế $y = 0$, ta sử dụng hàm loss $L = -\log(1 - \hat{y})$.

Toàn bộ phân tích ở trên được minh họa rõ ràng tại Hình 24, cho thấy hàm Binary Cross-Entropy hoạt động chính xác như chúng ta mong muốn.

9.5.2. Hàm tổng quát

Hàm loss $L(w, b)$ tổng quát cho toàn bộ m mẫu dữ liệu đơn giản là giá trị trung bình của tất cả các loss L_i trên từng mẫu:

$$L(w, b) = \frac{1}{m} \sum_{i=1}^m L_i = -\frac{1}{m} \sum_{i=1}^m (y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)).$$

Không giống hàm MSE vốn trở nên non-convex khi kết hợp với Sigmoid, hàm loss $L(w, b)$ sử dụng BCE này được chứng minh là một hàm convex.

Nhờ đặc điểm này, Gradient Descent không còn bị kẹt ở cực tiểu địa phương như khi dùng MSE. Với hàm lồi, mọi điểm dừng đều là cực tiểu toàn cục.

Cần lưu ý thêm một trường hợp. Khi hai lớp dữ liệu tách được hoàn toàn bằng một đường thẳng, cực tiểu của BCE không đạt được tại giá trị hữu hạn. Khi đó w và b tiếp tục tăng về độ lớn mà không dừng lại, dù loss vẫn giảm đều về 0. Ta sẽ thấy đúng hiện tượng này ở ví dụ thực hành phía sau.

9.6. Gradient Descent cho Logistic Regression

Sau khi đã có hàm loss $L(w, b)$, mục tiêu của chúng ta là tìm w và b sao cho L nhỏ nhất. Chúng ta lại sử dụng thuật toán quen thuộc là **Gradient Descent**.

Để áp dụng Gradient Descent, chúng ta cần tính đạo hàm riêng của L theo w và b . Quá trình tính toán đạo hàm này sử dụng quy tắc chuỗi vì các hàm được lồng vào nhau:

- Hàm $L(w, b)$ phụ thuộc vào $\hat{y}_i$.
- Xác suất $\hat{y}_i$ phụ thuộc vào z_i .
- Giá trị z_i phụ thuộc vào w và b .

9.6.1. Chi tiết phép toán đạo hàm

Ta có các công thức:

$$L = -\frac{1}{m} \sum_{i=1}^m [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)],$$

$$\hat{y}_i = g(z_i) = \frac{1}{1 + e^{-z_i}},$$

$$z_i = wx_i + b.$$

Vì hàm L là trung bình của các L_i , chúng ta có thể tập trung tính đạo hàm cho một mẫu L_i trước, sau đó lấy trung bình $1/m$ sau.

Mục tiêu 1: tìm $\frac{\partial L}{\partial w}$

Chúng ta sẽ áp dụng quy tắc chuỗi:

$$\frac{\partial L_i}{\partial w} = \frac{\partial L_i}{\partial \hat{y}_i} \cdot \frac{\partial \hat{y}_i}{\partial z_i} \cdot \frac{\partial z_i}{\partial w}.$$

Ta sẽ chia nhỏ và giải quyết từng thành phần:

1. **Phần 1:** $\frac{\partial L_i}{\partial \hat{y}_i}$ (Đạo hàm của Loss theo dự đoán)

$$L_i = -[y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)].$$

Lấy đạo hàm L_i theo $\hat{y}_i$:

$$\frac{\partial L_i}{\partial \hat{y}_i} = - \left[y_i \cdot \frac{1}{\hat{y}_i} + (1 - y_i) \cdot \frac{1}{1 - \hat{y}_i} \cdot (-1) \right] = - \left[\frac{y_i}{\hat{y}_i} - \frac{1 - y_i}{1 - \hat{y}_i} \right].$$

Quy đồng mẫu số và rút gọn:

$$\begin{aligned} \frac{\partial L_i}{\partial \hat{y}_i} &= \frac{-(y_i(1 - \hat{y}_i) - \hat{y}_i(1 - y_i))}{\hat{y}_i(1 - \hat{y}_i)} = \frac{-(y_i - y_i\hat{y}_i - \hat{y}_i + y_i\hat{y}_i)}{\hat{y}_i(1 - \hat{y}_i)} \\ &= \frac{\partial L_i}{\partial \hat{y}_i} = \frac{-(y_i - \hat{y}_i)}{\hat{y}_i(1 - \hat{y}_i)} = \frac{\hat{y}_i - y_i}{\hat{y}_i(1 - \hat{y}_i)}. \end{aligned}$$

2. **Phần 2:** $\frac{\partial \hat{y}_i}{\partial z_i}$ (Đạo hàm của hàm Sigmoid)

Đạo hàm của hàm Sigmoid viết lại được qua chính $g(z)$:

$$\hat{y}_i = g(z_i) = (1 + e^{-z_i})^{-1},$$

$$\frac{\partial \hat{y}_i}{\partial z_i} = -1 \cdot (1 + e^{-z_i})^{-2} \cdot (e^{-z_i} \cdot -1) = \frac{e^{-z_i}}{(1 + e^{-z_i})^2}.$$

Chúng ta có thể biến đổi nó một cách thông minh:

$$\begin{aligned} &= \left(\frac{1}{1 + e^{-z_i}} \right) \cdot \left(\frac{e^{-z_i}}{1 + e^{-z_i}} \right) = \left(\frac{1}{1 + e^{-z_i}} \right) \cdot \left(\frac{1 + e^{-z_i} - 1}{1 + e^{-z_i}} \right), \\ &= \left(\frac{1}{1 + e^{-z_i}} \right) \cdot \left(1 - \frac{1}{1 + e^{-z_i}} \right). \end{aligned}$$

Vì $\hat{y}_i = g(z_i) = \frac{1}{1 + e^{-z_i}}$, nên ta có kết quả:

$$\frac{\partial \hat{y}_i}{\partial z_i} = g(z_i) \cdot (1 - g(z_i)) = \hat{y}_i(1 - \hat{y}_i).$$

3. **Phần 3:** $\frac{\partial z_i}{\partial w}$ (Đạo hàm của phần tuyến tính)

$$z_i = wx_i + b,$$

$$\frac{\partial z_i}{\partial w} = x_i.$$

Kết hợp lại: Bây giờ, chúng ta nhân cả 3 phần lại với nhau:

$$\frac{\partial L_i}{\partial w} = \left(\frac{\partial L_i}{\partial \hat{y}_i} \right) \cdot \left(\frac{\partial \hat{y}_i}{\partial z_i} \right) \cdot \left(\frac{\partial z_i}{\partial w} \right),$$

$$\frac{\partial L_i}{\partial w} = \left(\frac{\hat{y}_i - y_i}{\hat{y}_i(1 - \hat{y}_i)} \right) \cdot (\hat{y}_i(1 - \hat{y}_i)) \cdot (x_i).$$

Đến đây, thành phần $\hat{y}_i(1 - \hat{y}_i)$ ở tử số và mẫu số triệt tiêu lẫn nhau.

$$\frac{\partial L_i}{\partial w} = (\hat{y}_i - y_i)x_i.$$

Đây mới chỉ là đạo hàm cho một mẫu L_i . Để có đạo hàm cho hàm L , ta chỉ cần lấy trung bình trên tất cả m mẫu:

$$\frac{\partial L}{\partial w} = \frac{1}{m} \sum_{i=1}^m \frac{\partial L_i}{\partial w} = \frac{1}{m} \sum_{i=1}^m (\hat{y}_i - y_i)x_i.$$

Mục tiêu 2: Tìm $\frac{\partial L}{\partial b}$

Quá trình này hoàn toàn tương tự, chỉ khác ở Phần 3:

$$\frac{\partial z_i}{\partial b} = 1.$$

Khi kết hợp lại:

$$\frac{\partial L_i}{\partial b} = \left(\frac{\hat{y}_i - y_i}{\hat{y}_i(1 - \hat{y}_i)} \right) \cdot (\hat{y}_i(1 - \hat{y}_i)) \cdot (1) = (\hat{y}_i - y_i).$$

Lấy trung bình trên m mẫu, ta có:

$$\frac{\partial L}{\partial b} = \frac{1}{m} \sum_{i=1}^m \frac{\partial L_i}{\partial b} = \frac{1}{m} \sum_{i=1}^m (\hat{y}_i - y_i).$$

9.6.2. Kết luận

Sau khi rút gọn tất cả các phép toán phức tạp, kết quả cuối cùng lại khá đơn giản:

$$\frac{\partial L}{\partial w} = \frac{1}{m} \sum_{i=1}^m (\hat{y}_i - y_i)x_i,$$

$$\frac{\partial L}{\partial b} = \frac{1}{m} \sum_{i=1}^m (\hat{y}_i - y_i).$$

Nếu bạn thấy quen thuộc thì đó là vì các công thức đạo hàm này trông giống hệt như công thức đạo hàm của Linear Regression. Đây không phải sự trùng hợp. BCE được chọn đúng để ghép với Sigmoid, nên thành phần $\hat{y}_i(1 - \hat{y}_i)$ triệt tiêu và gradient rút gọn về cùng một dạng.

i So sánh Gradient Descent: Linear vs. Logistic

Cả hai mô hình đều có chung một công thức cập nhật gradient về mặt hình thức:

$$\text{gradient} \sim (\text{Dự đoán} - \text{Thực tế}) \times \text{Đầu vào}$$

Tuy nhiên, có một điểm khác biệt cốt lõi nằm ở định nghĩa của dự đoán ($\hat{y}_i$):

- **Trong Linear Regression:**

$$\hat{y}_i = wx_i + b.$$

Giá trị $\hat{y}_i$ là một số thực bất kỳ (ví dụ: $\hat{y}_i = 105.5$).

- **Trong Logistic Regression:**

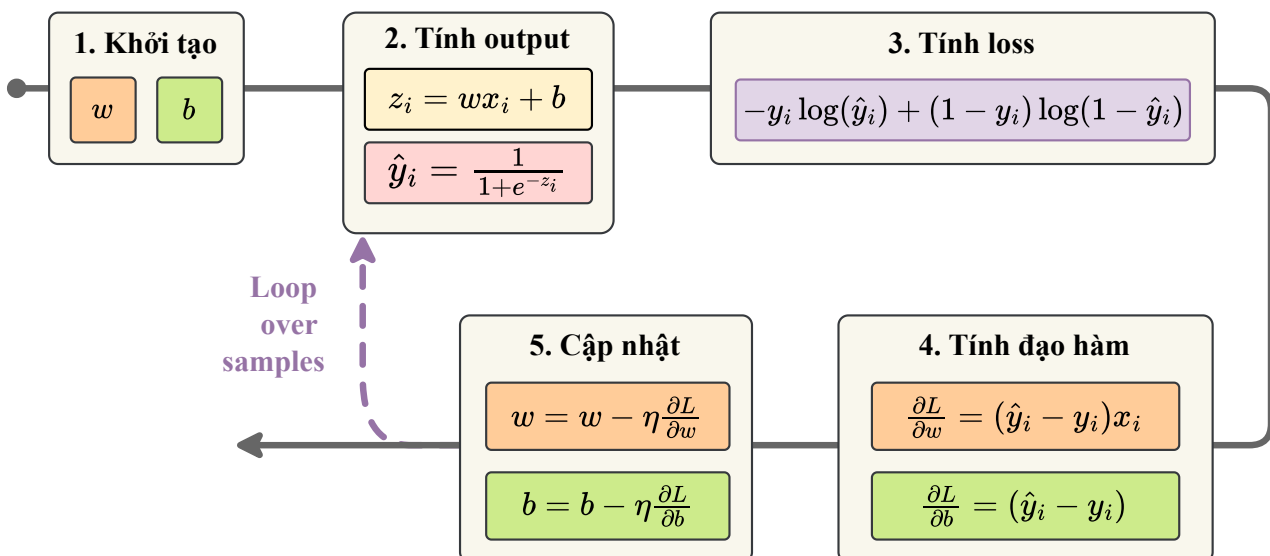
$$\hat{y}_i = g(wx_i + b) = \frac{1}{1 + e^{-(wx_i + b)}}.$$

Giá trị $\hat{y}_i$ là một xác suất luôn nằm trong khoảng $(0, 1)$ (ví dụ: $\hat{y}_i = 0.95$).

Mặc dù công thức đạo hàm trông giống nhau, nhưng giá trị của $(\hat{y}_i - y_i)$ được tính hoàn toàn khác nhau. Trong Linear, đó là $(wx_i + b) - y_i$. Trong Logistic, đó là $g(wx_i + b) - y_i$. Sự khác biệt trong cách tính $\hat{y}_i$ này dẫn đến quá trình hội tụ hoàn toàn khác nhau.

9.7. Quy trình huấn luyện

Quá trình huấn luyện cho Logistic Regression về cơ bản khá giống với Linear Regression, khi cũng sử dụng Batch Gradient Descent để tối ưu các tham số w và b . Toàn bộ các bước được minh họa trong Hình 25.



Hình 25: Quy trình tổng quát huấn luyện mô hình Logistic Regression.

1. Khởi tạo tham số

- Khởi tạo giá trị ban đầu cho w và b (ví dụ: $w = 0, b = 0$).
- Chọn các siêu tham số:
 - Learning rate η (ví dụ: 0.01).
 - Số vòng lặp (epochs), là số lần mô hình sẽ học qua toàn bộ m mẫu dữ liệu.

2. Lặp lại quá trình huấn luyện

Lặp theo số *epoch* đã chọn:

(a) Tính dự đoán $\hat{y}_i$ cho **tất cả** m mẫu:

- Tính đầu vào tuyến tính: $z_i = wx_i + b$.
- Tính xác suất dự đoán: $\hat{y}_i = g(z_i) = \frac{1}{1+e^{-z_i}}$.

(b) Tính hàm loss L trên toàn bộ tập dữ liệu:

$$L = -\frac{1}{m} \sum_{i=1}^m [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)].$$

(c) Tính đạo hàm trung bình trên toàn bộ m mẫu (Gradient):

$$\frac{\partial L}{\partial w} = \frac{1}{m} \sum_{i=1}^m (\hat{y}_i - y_i) x_i, \quad \frac{\partial L}{\partial b} = \frac{1}{m} \sum_{i=1}^m (\hat{y}_i - y_i).$$

(d) Cập nhật w và b đồng thời (Gradient Descent):

$$w = w - \eta \cdot \frac{\partial L}{\partial w}, \quad b = b - \eta \cdot \frac{\partial L}{\partial b}.$$

3. Kết thúc

Sau khi hoàn thành, chúng ta thu được các giá trị w và b đã được tối ưu.

Như vậy, bằng cách lặp quy trình trên, mô hình sẽ dần học được các tham số tối ưu. Sau mỗi epoch, giá trị của hàm loss $L(w, b)$ sẽ giảm xuống, và đường biên quyết định $wx + b = 0$ sẽ ngày càng di chuyển về vị trí tốt nhất để phân chia hai lớp dữ liệu 0 và 1.

Phần tiếp theo áp dụng toàn bộ lý thuyết này vào một ví dụ thực hành cụ thể.

10. Thực hành Logistic Regression

10.1. Phát biểu bài toán

Xét bài toán ví dụ dự đoán sinh viên thi Đậu (1) hay Trượt (0) dựa trên số giờ ôn tập. Chúng ta sẽ lấy ra 6 điểm dữ liệu ($m = 6$) để thực hành tính toán:

Sinh viên (i)	Số giờ học (x_i)	Kết quả (y_i)
1	1.0	0
2	2.5	0
3	4.0	0
4	6.0	1
5	7.0	1
6	8.0	1

Bảng 1: Bộ dữ liệu mẫu ($m = 6$) về kết quả thi.

Mục tiêu: Chúng ta sẽ thực hiện tính toán một vòng lặp (epoch) của Batch Gradient Descent để tìm ra các giá trị w và b mới, bắt đầu từ giá trị khởi tạo.

10.2. Chi tiết tính toán

Chúng ta sẽ tuân thủ quy trình huấn luyện đã được tóm tắt ở Mục 9.7..

1. Bước 1: Khởi tạo tham số

Để đơn giản, ta khởi tạo các giá trị ban đầu:

- Trọng số $w = 0$,
- Bias $b = 0$.

Đồng thời, ta chọn siêu tham số:

- Learning rate $\eta = 0.2$.

2. Bước 2: Lặp lại quá trình huấn luyện (Thực hiện Epoch 1)

Chúng ta thực hiện các bước tính toán cho vòng lặp đầu tiên.

(a) Bước 2a: Tính dự đoán $\hat{y}_i$ cho tất cả $m = 6$ mẫu

Đầu tiên, ta tính giá trị tuyến tính $z_i = wx_i + b$. Vì $w = 0$ và $b = 0$, nên $z_i = (0 \cdot x_i) + 0 = 0$ cho tất cả các mẫu $i = 1, \dots, 6$.

Tiếp theo, ta tính xác suất dự đoán $\hat{y}_i$ bằng hàm Sigmoid:

$$\hat{y}_i = g(z_i) = g(0) = \frac{1}{1 + e^{-0}} = \frac{1}{1 + 1} = 0.5.$$

Diễn giải: Tất cả 6 sinh viên đều có dự đoán xác suất Đậu là 0.5 (50%). Điều này hợp lý vì mô hình được khởi tạo $w = 0, b = 0$ nên nó hoàn toàn không biết gì và đưa ra cùng một xác suất 0.5 cho mọi trường hợp.

(b) **Bước 2b: Tính hàm loss L**

Ta sử dụng công thức Binary Cross-Entropy tổng quát:

$$L = -\frac{1}{m} \sum_{i=1}^m [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)].$$

Vì ở bước 2a, ta có $\hat{y}_i = 0.5$ cho tất cả các mẫu, ta thay vào:

$$L = -\frac{1}{6} \sum_{i=1}^6 [y_i \log(0.5) + (1 - y_i) \log(0.5)]$$

Ta nhóm $\log(0.5)$ lại:

$$\begin{aligned} L &= -\frac{1}{6} \sum_{i=1}^6 [(y_i + 1 - y_i) \log(0.5)] = -\frac{1}{6} \sum_{i=1}^6 [1 \cdot \log(0.5)], \\ L &= -\frac{1}{6} [6 \cdot \log(0.5)] = -\log(0.5) = \mathbf{0.6931}. \end{aligned}$$

(c) **Bước 2c: Tính đạo hàm trung bình**

Bây giờ, chúng ta cần tính $\frac{\partial L}{\partial w}$ và $\frac{\partial L}{\partial b}$. Trước tiên chúng ta cần tính các thành phần sai số $(\hat{y}_i - y_i)$ và $(\hat{y}_i - y_i)x_i$ cho từng mẫu.

Để cho trực quan, chúng ta có thể tổng hợp các giá trị này trong một bảng:

Mẫu (i)	x_i	y_i	$\hat{y}_i$	Sai số $(\hat{y}_i - y_i)$	Thành phần $w (\hat{y}_i - y_i)x_i$
1	1.0	0	0.5	$0.5 - 0 = \mathbf{0.5}$	$0.5 \times 1.0 = \mathbf{0.5}$
2	2.5	0	0.5	$0.5 - 0 = \mathbf{0.5}$	$0.5 \times 2.5 = \mathbf{1.25}$
3	4.0	0	0.5	$0.5 - 0 = \mathbf{0.5}$	$0.5 \times 4.0 = \mathbf{2.0}$
4	6.0	1	0.5	$0.5 - 1 = \mathbf{-0.5}$	$-0.5 \times 6.0 = \mathbf{-3.0}$
5	7.0	1	0.5	$0.5 - 1 = \mathbf{-0.5}$	$-0.5 \times 7.0 = \mathbf{-3.5}$
6	8.0	1	0.5	$0.5 - 1 = \mathbf{-0.5}$	$-0.5 \times 8.0 = \mathbf{-4.0}$
Tổng Σ	-	-	-	0.0	-6.75

Bảng 2: Tính toán các thành phần đạo hàm cho Epoch 1.

Dựa vào cột tổng (Σ) của Bảng 2, ta có:

- $\sum_{i=1}^m (\hat{y}_i - y_i) = 0.0$,
- $\sum_{i=1}^m (\hat{y}_i - y_i)x_i = (0.5 + 1.25 + 2.0) - (3.0 + 3.5 + 4.0) = 3.75 - 10.5 = \mathbf{-6.75}$.

Giờ ta có thể dễ dàng tính các đạo hàm trung bình (với $m = 6$):

$$\begin{aligned} \frac{\partial L}{\partial b} &= \frac{1}{m} \sum_{i=1}^m (\hat{y}_i - y_i) = \frac{1}{6}(0.0) = \mathbf{0}, \\ \frac{\partial L}{\partial w} &= \frac{1}{m} \sum_{i=1}^m (\hat{y}_i - y_i)x_i = \frac{1}{6}(-6.75) = \mathbf{-1.125}. \end{aligned}$$

(d) **Bước 2d: Cập nhật w và b**

Chúng ta sử dụng công thức cập nhật Gradient Descent với $\eta = 0.2$:

$$w_{\text{mới}} = w - \eta \cdot \frac{\partial L}{\partial w} = 0 - (0.2) \cdot (-1.125) = \mathbf{0.225},$$

$$b_{\text{mới}} = b - \eta \cdot \frac{\partial L}{\partial b} = 0 - (0.2) \cdot (0) = \mathbf{0}.$$

3. Bước 3: Kết thúc Epoch 1

Sau khi hoàn thành, các tham số mới của mô hình là:

- $w = 0.225$,
- $b = 0$.

10.3. Phân tích kết quả epoch 1

Hãy cùng xem mô hình đã học được gì chỉ sau một bước cập nhật:

- Giá trị w đã thay đổi từ 0 lên **0.225**. Đạo hàm $\frac{\partial L}{\partial w}$ là **-1.125** (âm), nghĩa là tăng w sẽ làm giảm loss. Thuật toán Gradient Descent đã cập nhật đúng theo hướng đó bằng cách đi ngược dấu đạo hàm.
- Giá trị w mới là số dương, cho thấy mô hình đang bắt đầu học được rằng x (số giờ học) càng *tăng* thì xác suất Đậu cũng càng *tăng*.

Dự đoán lại với tham số mới:

Dùng $w \approx 0.225$ và $b = 0$ để dự đoán cho sinh viên 4 (người học $x_4 = 6.0$ giờ và có nhãn $y_4 = 1$):

1. Tính $z_4^{\text{mới}}$:

$$z_4^{\text{mới}} = w_{\text{mới}} \cdot x_4 + b_{\text{mới}} \approx (0.225 \cdot 6.0) + 0 = \mathbf{1.35}.$$

2. Tính $\hat{y}_4^{\text{mới}}$:

$$\hat{y}_4^{\text{mới}} = g(z_4^{\text{mới}}) = g(1.35) = \frac{1}{1 + e^{-1.35}} \approx \frac{1}{1 + 0.2592} \approx \mathbf{0.7941}.$$

Dự đoán mới là **0.7941** (79.41%), đã nhích lên đáng kể so với dự đoán 0.5 ban đầu, và tiến gần hơn đến nhãn thực tế là $y_4 = 1$.

Sau một epoch, loss đã giảm từ 0.6931 xuống 0.6064 và dự đoán tiến gần nhãn thực tế hơn. Nếu lặp lại quá trình này, w và b sẽ tiếp tục được điều chỉnh, cho đến khi mô hình tìm được đường cong phù hợp để phân chia dữ liệu.

10.4. Phân tích kết quả huấn luyện qua các epoch

Nếu chúng ta lặp lại quá trình này hàng nghìn lần, w và b sẽ tiếp tục được điều chỉnh. Bảng 3 tóm tắt quá trình học của mô hình qua 10000 epochs, với dự đoán cho mẫu $x = 6.0$.

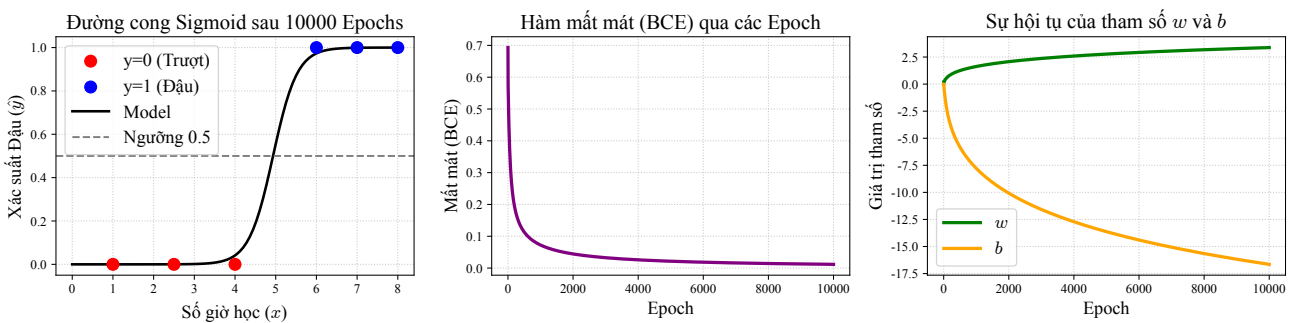
Epoch	Trọng số (w)	Bias (b)	Loss (BCE)	Dự đoán $\hat{y}$ ($x = 6.0$)
0	0.0000	0.0000	0.6931	0.5000
1	0.2250	0.0000	0.6064	0.7941
2	0.1776	-0.0462	0.5916	0.7349
5	0.1956	-0.1540	0.5718	0.7349
10	0.2260	-0.3264	0.5413	0.7368
50	0.4280	-1.4464	0.3783	0.7542
500	1.2504	-5.8236	0.1130	0.8428
2000	2.0740	-10.0757	0.0444	0.9144
10000	3.3737	-16.6475	0.0117	0.9733

Bảng 3: Tóm tắt quá trình huấn luyện qua 10000 epochs.

Phân tích (từ Bảng 3):

- **Trọng số w :** Tăng từ 0 lên ≈ 3.3737 , chỉ giảm nhẹ một lần ở epoch 2 rồi tăng đều. Mô hình học rất rõ ràng rằng w dương (số giờ học tăng thì khả năng đậu tăng).
- **Bias b :** Giảm dần từ 0 xuống ≈ -16.6475 , được điều chỉnh để tìm ra đường ranh giới quyết định phù hợp nhất với dữ liệu.
- **Loss (BCE):** Giảm từ 0.6931 xuống còn **0.0117**. Đây là bằng chứng rõ ràng nhất cho thấy mô hình đang học tốt và hội tụ.
- **Dự đoán $\hat{y}$ ($x = 6.0$):** Dự đoán cho sinh viên học 6.0 giờ tăng từ 50% lên 79.4% (sau 1 epoch), và đạt ≈ 0.9733 (97.3%) ở epoch 10000, ngày càng tự tin hơn về nhận thực tế $y = 1$.

Để trực quan hoá toàn bộ quá trình, Hình 26 biểu diễn các kết quả huấn luyện một cách rõ ràng:



Hình 26: Trực quan hoá quá trình huấn luyện Logistic Regression sau 10000 epochs.

Phân tích Hình 26:

- **(Biểu đồ giữa) Hàm loss (BCE):** Hàm loss giảm gần hết trong vài trăm epoch đầu (còn 0.1130 ở epoch 500), sau đó giảm chậm dần về gần 0. Diễn biến này khớp với cột “Loss (BCE)” trong Bảng 3.
- **(Biểu đồ phải) Sự hội tụ của tham số:** Biểu đồ này trực quan hoá hai cột Trọng số (w) và Bias (b) của bảng. Ta thấy w (màu xanh lá) tăng dần từ 0 lên giá trị dương và b (màu cam) giảm dần từ 0 xuống giá trị âm. Tốc độ thay đổi mạnh nhất ở các epoch đầu rồi chậm lại, nhưng đến epoch 10000 cả hai vẫn chưa dừng hẳn. Đây chính là trường hợp dữ liệu tách được tuyến tính đã nêu ở phần Binary Cross-Entropy.
- **(Biểu đồ trái) Đường cong Sigmoid cuối cùng:** Đây là kết quả sau 10000 epochs, khi $w \approx 3.37$ và $b \approx -16.65$. Đường cong Sigmoid (màu đen) đã uốn theo đúng hai nhóm dữ liệu, với dự đoán $\hat{y} \approx 0$ cho các điểm $y = 0$ (Trượt, màu đỏ) và $\hat{y} \approx 1$ cho các điểm $y = 1$ (Đậu, màu xanh).

11. Xây dựng mô hình Logistic Regression cho bài toán Titanic

Ở phần trên, chúng ta đã tính tay một epoch trên bộ dữ liệu sáu sinh viên để thấy rõ từng bước. Trong phần này, chúng ta sẽ áp dụng đúng quy trình đó lên một bộ dữ liệu thật với nhiều đặc trưng, đồng thời bổ sung hai thứ mà bộ dữ liệu nhỏ chưa cần đến là chuẩn hoá đặc trưng và tập validation.

Phát biểu bài toán: Cho bộ dữ liệu hành khách tàu Titanic, hãy xây dựng mô hình Logistic Regression để dự đoán một hành khách có sống sót hay không.

Giới thiệu về tập data: Bộ dữ liệu gồm 891 mẫu, 8 đặc trưng đầu vào và 2 lớp. Cột Survived nhận giá trị 1 nếu hành khách sống sót và 0 nếu ngược lại.

STT	Đặc trưng	Ý nghĩa
1	Pclass	Hạng vé, nhận giá trị 1, 2 hoặc 3
2	Sex	Giới tính, đã mã hoá thành 0 và 1
3	Age	Tuổi
4	SibSp	Số anh chị em và vợ chồng đi cùng
5	Parch	Số cha mẹ và con cái đi cùng
6	Fare	Giá vé
7	Embarked	Cảng lên tàu, đã mã hoá thành số
8	Title	Danh xưng trích từ tên, đã mã hoá thành số

Bảng 4: Tám đặc trưng đầu vào của bộ dữ liệu Titanic.

11.1. Bước 1: Tải và đọc dữ liệu

```
1 !gdown 1KehLp3sWEtKtSARYoV7luPqDhtRPj10R
2 !unzip titanic.zip
```

```
1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4
5 from sklearn.model_selection import train_test_split
6 from sklearn.preprocessing import StandardScaler
7
8 dataset_path = 'titanic_modified_dataset.csv'
9 df = pd.read_csv(dataset_path, index_col='PassengerId')
10 df
```

Tham số `index_col='PassengerId'` đặt cột mã hành khách làm chỉ mục, nhờ đó cột này không bị hiểu nhầm thành một đặc trưng đầu vào.

11.2. Bước 2: Khám phá dữ liệu

```
1 df.info()
2 df.describe()
```

Hai lệnh này cho biết hai điều cần kiểm tra trước khi huấn luyện. Thứ nhất, cả 891 dòng đều đủ giá trị ở mọi cột nên không phải xử lý dữ liệu thiếu. Thứ hai, các đặc trưng có thang đo rất khác nhau, chẳng hạn `Pclass` chỉ nhận giá trị từ 1 đến 3 trong khi `Fare` trải từ 0 đến hơn 500 và `Age` có trung bình 29.36. Chênh lệch này chính là lý do phải chuẩn hoá ở Bước 5.

11.3. Bước 3: Tách X , y và thêm cột bias

```
1 dataset_arr = df.to_numpy().astype(np.float64)
2 X, y = dataset_arr[:, :-1], dataset_arr[:, -1]
3
4 print(X.shape)  # (891, 8)
```

Tám cột đầu trở thành ma trận đặc trưng X , cột cuối là nhãn y .

Ở các phần trước, Bias b được viết tách riêng khỏi trọng số. Khi làm việc với ma trận, cách gọn hơn là ghép b vào chung vector tham số θ . Muốn vậy, ta thêm vào X một cột toàn số 1 ở phía trước, khi đó θ_0 đóng đúng vai trò của Bias vì nó luôn được nhân với 1.

```
1 intercept = np.ones((X.shape[0], 1))
2 X_b = np.concatenate((intercept, X), axis=1)
3
4 print(X_b.shape)  # (891, 9)
```

Sau bước này X_b có 9 cột, gồm 1 cột bias và 8 cột đặc trưng, nên θ cũng sẽ có 9 phần tử.

11.4. Bước 4: Chia tập train, validation và test

```

1 val_size = 0.2
2 test_size = 0.125
3 random_state = 2
4 is_shuffle = True
5
6 X_train, X_val, y_train, y_val = train_test_split(
7     X_b, y, test_size=val_size,
8     random_state=random_state, shuffle=is_shuffle
9 )
10
11 X_train, X_test, y_train, y_test = train_test_split(
12     X_train, y_train, test_size=test_size,
13     random_state=random_state, shuffle=is_shuffle
14 )

```

Lần gọi thứ nhất tách 20% dữ liệu ra làm tập validation. Lần gọi thứ hai tách tiếp 12.5% của phần còn lại ra làm tập test. Kết quả là 623 mẫu train, 179 mẫu validation và 89 mẫu test.

Bài toán này dùng ba tập thay vì hai như các phần trước. Tập validation cho phép theo dõi mô hình trên dữ liệu chưa học trong suốt quá trình huấn luyện và chọn siêu tham số, còn tập test chỉ được dùng đúng một lần ở cuối để con số đánh giá không bị ảnh hưởng bởi các lựa chọn đã thực hiện.

11.5. Bước 5: Chuẩn hoá đặc trưng

```

1 normalizer = StandardScaler()
2
3 X_train[:, 1:] = normalizer.fit_transform(X_train[:, 1:])
4 X_val[:, 1:] = normalizer.transform(X_val[:, 1:])
5 X_test[:, 1:] = normalizer.transform(X_test[:, 1:])

```

Hai chi tiết cần chú ý ở đoạn code này:

- Chỉ số `[:, 1:]` bỏ qua cột đầu tiên. Cột đó là cột bias toàn số 1, nếu chuẩn hoá thì độ lệch chuẩn bằng 0 và phép chia sẽ hỏng.
- `StandardScaler` được `fit` trên tập train, sau đó chỉ `transform` cho tập validation và tập test. Nếu `fit` trên cả ba tập thì trung bình và độ lệch chuẩn của dữ liệu chưa từng thấy sẽ lọt vào bước chuẩn hoá, khiến kết quả đánh giá tốt hơn so với thực tế.

11.6. Bước 6: Huấn luyện mô hình

Trước khi viết vòng lặp huấn luyện, chúng ta cần sáu hàm. Năm hàm đầu chính là năm bước đã trình bày ở Mục 9.7., hàm thứ sáu dùng để theo dõi kết quả.

Yêu cầu 11: Hoàn thành hàm `sigmoid()` và hàm `predict()`.

```

1 def sigmoid(z):
2     ***Your code here***
3

```

```

4 def predict(X, theta):
5     dot_product = ***Your code here***
6     y_hat = ***Your code here***
7
8     return y_hat

```

Hàm `predict()` cần tính $z_i = \mathbf{x}_i^\top \boldsymbol{\theta}$ cho toàn bộ các mẫu bằng một phép nhân ma trận giữa $\mathbf{X}$ và $\boldsymbol{\theta}$, thay cho vòng lặp qua từng mẫu. Kết quả sau đó được đưa qua hàm `sigmoid()` để thu được xác suất $\hat{y}$.

☰ **Yêu cầu 12:** Hoàn thành hàm `compute_loss()` tính Binary Cross-Entropy.

```

1 def compute_loss(y_hat, y):
2     y_hat = np.clip(y_hat, 1e-7, 1 - 1e-7)
3
4     return ***Your code here***

```

Hàm `np.clip` chặn $\hat{y}$ trong khoảng $[10^{-7}, 1 - 10^{-7}]$. Không có bước này, khi mô hình dự đoán quá tự tin và $\hat{y}$ chạm đúng 0 hoặc 1 thì $\log(0)$ cho ra vô cực và loss trở thành `nan`.

☰ **Yêu cầu 13:** Hoàn thành hàm `compute_gradient()` và hàm `update_theta()`.

```

1 def compute_gradient(X, y, y_hat):
2     return ***Your code here***
3
4 def update_theta(theta, gradient, lr):
5     return ***Your code here***

```

Hàm `compute_gradient()` cài đặt công thức $\frac{\partial L}{\partial \boldsymbol{\theta}} = \frac{1}{m} \mathbf{X}^\top (\hat{\mathbf{y}} - \mathbf{y})$ đã suy ra ở Mục 9.6., viết gọn dưới dạng ma trận. Hệ số m là số mẫu trong batch đang xét. Hàm `update_theta()` đi ngược hướng gradient một bước có độ dài bằng learning rate.

☰ **Yêu cầu 14:** Hoàn thành hàm `compute_accuracy()`.

```

1 def compute_accuracy(X, y, theta):
2     y_hat = ***Your code here***
3     acc = ***Your code here***
4
5     return acc

```

Để đổi xác suất $\hat{y}$ thành nhãn, áp dụng ngưỡng quyết định 0.5. Mẫu có $\hat{y} \geq 0.5$ nhận nhãn 1, các mẫu còn lại nhận nhãn 0. Accuracy là tỉ lệ nhãn dự đoán trùng với nhãn thực tế.

Tiếp theo là các siêu tham số và giá trị khởi tạo:

```

1 lr = 0.01
2 epochs = 100
3 batch_size = 16
4
5 np.random.seed(random_state)
6 theta = np.random.uniform(size=X_train.shape[1])

```

Với `batch_size` bằng 16 và 623 mẫu train, mỗi epoch có 39 lần cập nhật tham số. Đây là Mini-batch Gradient Descent, nằm giữa hai thái cực đã trình bày ở phần Linear Regression. Lệnh `np.random.seed` cố định bộ sinh số ngẫu nhiên để lần chạy sau cho ra đúng kết quả cũ.

Yêu cầu 15: Hoàn thành vòng lặp huấn luyện.

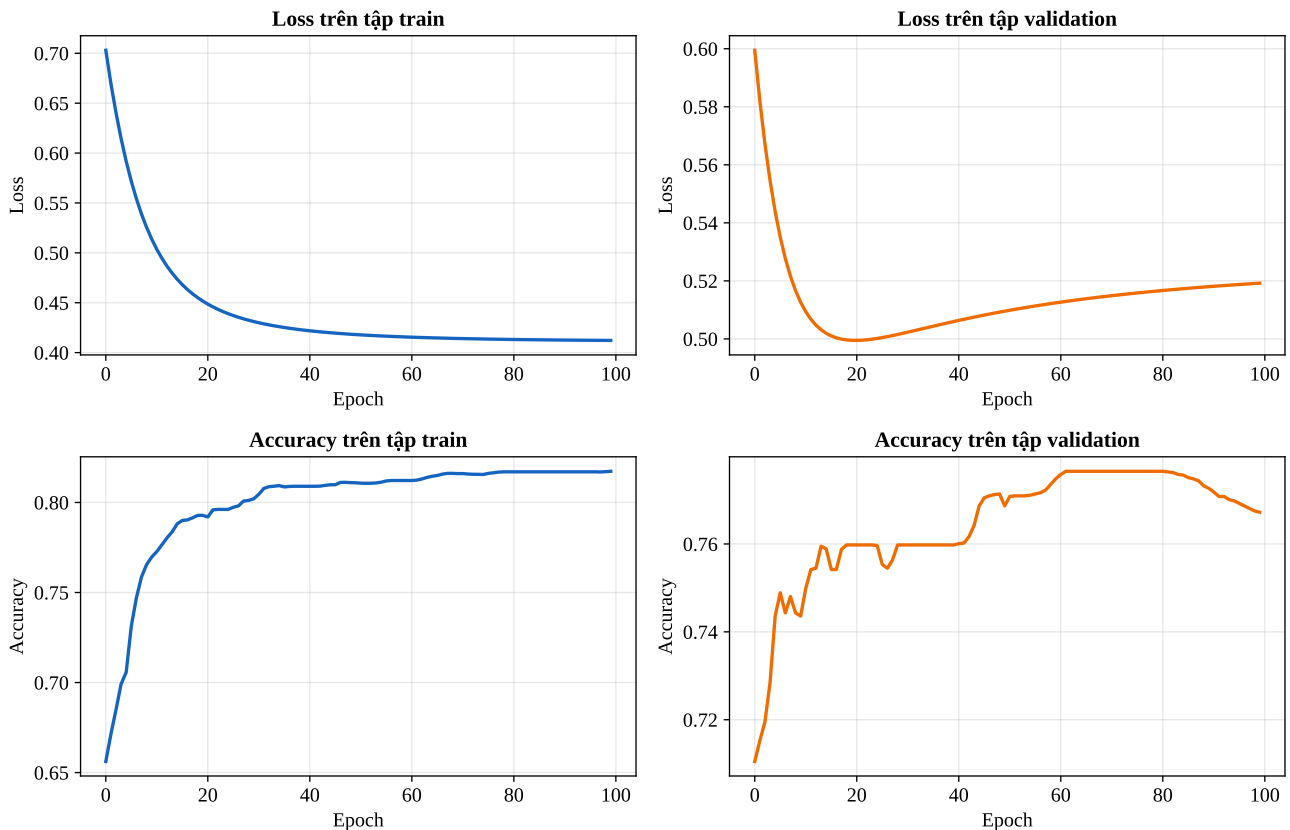
```

1 train_accs, train_losses = [], []
2 val_accs, val_losses = [], []
3
4 for epoch in range(epochs):
5     train_batch_losses, train_batch_accs = [], []
6     val_batch_losses, val_batch_accs = [], []
7
8     for i in range(0, X_train.shape[0], batch_size):
9         X_i = X_train[i:i+batch_size]
10        y_i = y_train[i:i+batch_size]
11
12        y_hat = ***Your code here***
13        train_loss = ***Your code here***
14        gradient = ***Your code here***
15        theta = ***Your code here***
16
17        train_batch_losses.append(train_loss)
18        train_batch_accs.append(compute_accuracy(X_train, y_train, theta))
19
20        y_val_hat = predict(X_val, theta)
21        val_batch_losses.append(compute_loss(y_val_hat, y_val))
22        val_batch_accs.append(compute_accuracy(X_val, y_val, theta))
23
24        train_losses.append(sum(train_batch_losses) / len(train_batch_losses))
25        val_losses.append(sum(val_batch_losses) / len(val_batch_losses))
26        train_accs.append(sum(train_batch_accs) / len(train_batch_accs))
27        val_accs.append(sum(val_batch_accs) / len(val_batch_accs))

```

Vòng lặp ngoài chạy qua các epoch, vòng lặp trong cắt tập train thành từng batch 16 mẫu. Bốn dòng ở giữa chính là bốn bước dự đoán, tính loss, tính gradient và cập nhật tham số. Các dòng còn lại chỉ ghi lại số liệu để vẽ biểu đồ, chúng không tham gia vào việc huấn luyện.

11.7. Bước 7: Đánh giá mô hình



Hình 27: Loss và accuracy trên tập train và tập validation qua 100 epoch.

Phân tích Hình 27:

- **Loss trên tập train** giảm đều từ 0.703 xuống 0.412 và gần như nằm ngang từ epoch 40 trở đi.
- **Loss trên tập validation** giảm từ 0.599 xuống thấp nhất khoảng 0.499 quanh epoch 20, sau đó tăng dần lên 0.519 ở epoch 100. Đây là dấu hiệu của overfitting nhẹ, khi mô hình bắt đầu khớp với các chi tiết riêng của tập train mà không còn tổng quát thêm được cho dữ liệu mới.
- **Accuracy** trên tập train tăng tới khoảng 0.817, còn trên tập validation dừng quanh 0.767. Khoảng cách này nhỏ, nên mức overfitting ở đây chưa nghiêm trọng.

Từ diễn biến trên, có thể dừng huấn luyện quanh epoch 20 để lấy mô hình có loss validation thấp nhất. Đây chính là công dụng của tập validation mà tập test không thể thay thế.

```

1 val_set_acc = compute_accuracy(X_val, y_val, theta)
2 test_set_acc = compute_accuracy(X_test, y_test, theta)
3
4 print('Evaluation on validation set:')
5 print(f'Accuracy: {val_set_acc}')
6

```

```
7 print('Evaluation on test set:')  
8 print(f'Accuracy: {test_set_acc}')
```

Kết quả thu được là 0.7709 trên tập validation và 0.7753 trên tập test. Hai con số gần nhau cho thấy mô hình hoạt động ổn định trên dữ liệu chưa từng thấy. Để biết con số này có tốt hay không, cần một mốc so sánh. Nếu đoán toàn bộ hành khách đều không sống sót thì accuracy bằng tỉ lệ lớp đa số của tập test là 0.5393. Mô hình đã cải thiện thêm 23.6 điểm phần trăm so với mốc đó.

12. Tài liệu tham khảo

- [1] A.-M. Legendre, *Nouvelles méthodes pour la détermination des orbites des comètes*. Paris, France: Firmin Didot, 1805, Appendix *Sur la méthode des moindres quarrés*, pp. 72-75.
- [2] C. F. Gauss, *Theoria motus corporum coelestium in sectionibus conicis solem ambientium*. Hamburg, Germany: Perthes und Besser, 1809.
- [3] F. Galton, “Regression towards mediocrity in hereditary stature”, *The Journal of the Anthropological Institute of Great Britain and Ireland*, vol. 15, pp. 246–263, 1886.
- [4] S. M. Stigler, *The History of Statistics: The Measurement of Uncertainty before 1900*. Cambridge, MA, USA: Belknap Press of Harvard University Press, 1986.
- [5] G. James et al., *An Introduction to Statistical Learning: with Applications in R* (Springer Texts in Statistics). New York, NY, USA: Springer, 2013.
- [6] S. Ruder, “An overview of gradient descent optimization algorithms”, *arXiv preprint arXiv:1609.04747*, 2016.
- [7] L. Bottou, “Large-scale machine learning with stochastic gradient descent”, in *Proceedings of COMPSTAT'2010*, Physica-Verlag HD, 2010, pp. 177–186.
- [8] V. H. Tip. “Bài 10: Logistic regression”, Accessed: Sep. 7, 2026. [Online]. Available: <https://machinelearningcoban.com/2017/01/27/logisticregression/>.
- [9] D. R. Cox, “The regression analysis of binary sequences”, *Journal of the Royal Statistical Society. Series B (Methodological)*, vol. 20, no. 2, pp. 215–242, 1958.
- [10] J. S. Cramer, “The origins of logistic regression”, University of Amsterdam, Tech. Rep., 2002, Tinbergen Institute Discussion Paper, TI 2002-119/4.
- [11] Y. S. Abu-Mostafa et al., *Learning From Data: A Short Course*. New York, NY, USA: AMLBook, 2012.
- [12] A. Ng, *Cs229 lecture notes: Part ii: Classification and logistic regression*, Course notes, Stanford University, 2008.

13. Phụ lục

1. **Resources:** Các file code gợi ý (hint) và bài giải (solution) có thể được tải tại [đây](#). (**Lưu ý:** Sau khi hết deadline của phần bài này, ad mới copy các nội dung bài giải vào đường dẫn).

2. Rubric:

Mục	Kiến thức	Đánh giá
1.	- Quy trình huấn luyện Linear Regression theo từng mẫu (Stochastic Gradient Descent) gồm khởi tạo trọng số, lấy mẫu, dự đoán, tính loss, tính gradient, cập nhật tham số. - Công thức toán cho từng bước với nhiều đặc trưng đầu vào (w_1, w_2, w_3, b).	- Trình bày đúng công thức dự đoán/loss/gradient/cập nhật cho từng bước. - Hiểu vòng lặp huấn luyện lặp qua từng mẫu dữ liệu.
2.	- Đọc và xử lý dữ liệu advertising.csv bằng Python/NumPy thuần. - Cài đặt các hàm predict, compute_loss, compute_gradient_wi/b, update_weight_wi/b theo từng mẫu.	- Hoàn thành đúng các hàm get_column/prepare_data. - Cài đặt đúng predict, compute_loss, compute_gradient, update_weight. - Huấn luyện được mô hình bằng implement_linear_regression và đọc đúng kết quả loss/trọng số.
3.	- Min-Max Scaling và Z-score Scaling (Standardization). - Phép biến đổi ngược (inverse transform) để khôi phục dữ liệu gốc.	- Hiểu công thức scale/inverse scale của cả hai phương pháp.
4.	- Ba biến thể Stochastic, Mini-batch và Batch Gradient Descent. - Cài đặt theo kiểu vector hoá với NumPy.	- Phân biệt được cách cập nhật tham số của ba biến thể. - Cài đặt được ba biến thể theo kiểu vector hoá.
5.	- Hiểu về cách áp dụng của thuật toán Linear Regression cho dữ liệu Time Series - Phân tích một số đặc trưng của dữ liệu Time Series	- Áp dụng Linear Regression cho bài toán dự đoán giá Bitcoin - Đánh giá hiệu quả trên những cài đặt khác nhau
6.	- Bài toán phân loại nhị phân và giới hạn của Linear Regression khi áp dụng cho phân loại. - Hàm Sigmoid, ngưỡng quyết định và đường biên quyết định. - Hàm loss Binary Cross-Entropy và tính lồi (convex) so với MSE. - Đạo hàm của BCE qua hàm Sigmoid.	- Giải thích được vì sao đầu ra tuyến tính không diễn giải được thành xác suất. - Diễn giải được $\hat{y}$ là $P(y = 1 x)$ và ngưỡng 0.5 tương ứng $z = 0$. - Nêu được lý do không dùng MSE cho Logistic Regression. - Suy ra được $\partial L / \partial w = \frac{1}{m} \sum (\hat{y}_i - y_i) x_i$.

7.	- Quy trình huấn luyện Logistic Regression bằng Batch Gradient Descent. - Tính tay một epoch trên bộ dữ liệu nhỏ. - Phân tích quá trình hội tụ qua các epoch.	- Thực hiện đúng các bước dự đoán, tính loss, tính gradient và cập nhật tham số. - Giải thích được ý nghĩa dấu của gradient và của w, b sau khi hội tụ. - Đọc được biểu đồ loss và biểu đồ hội tụ tham số.
8.	- Logistic Regression trên dữ liệu thật nhiều đặc trưng (Titanic). - Vai trò của chuẩn hoá đặc trưng và tập validation.	- Cài đặt đúng các hàm sigmoid, predict, BCE, gradient, accuracy và vòng lặp Mini-batch. - Đọc được biểu đồ loss và accuracy để đánh giá quá trình huấn luyện.

3. **Q&A:** Bạn có thể đặt thêm câu hỏi về nội dung bài đọc trong group Facebook hỏi đáp tại [đây](#). Tất cả câu hỏi sẽ được trả lời trong vòng tối đa 4 tiếng.

AIO_Family

Private group · 2.0K members



Hình 28: Hình ảnh group facebook AIO Q&A.